

## Chapter – 6 Natural Language Processing (NLP)

### Contents:

Overview of NLP

6.1 Language Models

6.2 Part-of-Speech (POS) Tagging

6.3 Grammar and Parsing

6.4 Complications of Real Natural Language

6.5 Word Embeddings

6.6 RNN for NLP

6.7 LSTMs for NLP

6.8 Sequence-to-Sequence Models

6.9 The Transformer Architecture- BERT, GPT, Attention

### **Overview of NLP:**

Natural Language Processing (NLP) is a subfield of artificial intelligence (AI) and linguistics that focuses on enabling computers to **understand**, **interpret**, **generate**, and **respond** to human language in a way that is both **meaningful** and **useful**.

It bridges the gap between human communication (natural language) and computer understanding.

NLP enables computers and digital devices to recognize, understand and generate text and speech by combining computational linguistics, the rule-based modeling of human language together with statistical modeling, machine learning and deep learning.

NLP research has helped enable the era of generative AI, from the communication skills of large language models (LLMs) to the ability of image generation models to understand requests.

NLP is already part of everyday life for many, powering search engines, prompting chatbots for customer service with spoken commands, voice-operated GPS systems and question-answering digital assistants on smartphones such as Amazon's Alexa, Apple's Siri and Microsoft's Cortana.

### Components of NLP:

1. Natural Language Understanding (NLU):

- NLU focuses on helping machines comprehend and interpret human language.
- It involves tasks like tokenization, part-of-speech tagging, named entity recognition, and semantic analysis to understand meaning, context, and intent behind the text.

2. Natural Language Generation (NLG):

- NLG is about enabling machines to produce human-like language.
- It takes structured data or machine-readable input and converts it into coherent (logical), natural-sounding text or speech used in chatbots, report generation, and text summarization.

### Why it is important?

The applications of NLP are vast and growing, impacting nearly every aspect of our digital lives.

It's the technology behind:

- Improved data analysis: enabling the extraction of insights from unstructured text data, such as customer reviews, social media posts and news articles.
- Enhanced Search: understand the intent behind user queries, providing more accurate and contextually relevant results.
- Powerful content generation: create human-like text eg. GPT-4
- Virtual Assistants (Siri, Alexa, Google Assistant): Interpreting your voice commands and responding appropriately.

- Machine Translation (Google Translate): Converting text or speech from one language to another.
- Spam Filters: Identifying and blocking unwanted emails.
- Sentiment Analysis: Determining the emotional tone of text (e.g., in customer reviews or social media).
- Chatbots and Customer Service: Automating interactions and providing quick answers.
- Text Summarization: Condensing long documents into shorter, coherent summaries.
- Speech Recognition: Converting spoken words into written text.
- Information Extraction: Pulling specific data points from unstructured text.

### Goals of NLP:

The primary goals of NLP are:

- To enable machines to understand natural language text or speech.
- To allow interaction between humans and machines using natural language.
- To convert human language into structured data for analysis and use in applications.

### Key Tasks Involved in NLP:

Some fundamental tasks involved in NLP include:

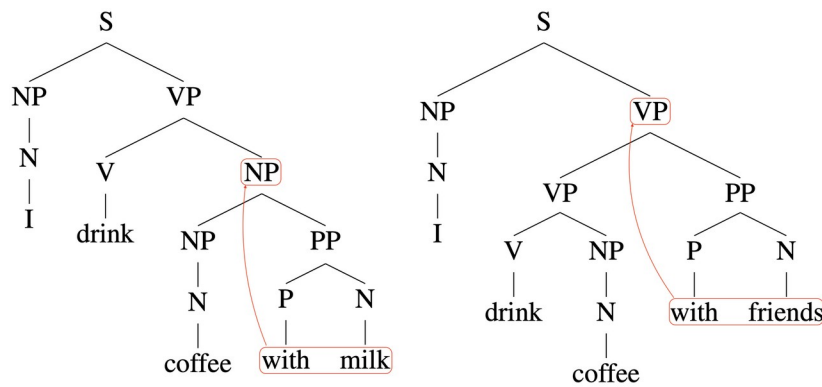
- Tokenization: Splitting text into words, phrases, symbols, or other meaningful elements called tokens.
- Part-of-Speech (POS) Tagging: Assigning word categories (e.g., noun, verb) to each token.
- Named Entity Recognition (NER): Identifying names of people, organizations, locations, etc.
- Parsing: Analyzing grammatical structure of a sentence. understand how words relate to each other : like which word is the subject, which is the verb, which words are objects, etc
- Sentiment Analysis: Determining the emotion or opinion expressed in a text.
- Machine Translation: Automatically translating text from one language to another.
- Text Summarization: Generating a concise version of a longer document.
- Speech Recognition: Converting spoken language into text.
- Question Answering and Chatbots: Building systems that can respond to user queries in natural language.

### Approaches to NLP:

- Natural Language Processing (NLP) combines computational linguistics with machine learning and deep learning to analyze and understand language.
- It involves two main types of analysis:
  - syntactic analysis, which uses grammar rules and parsing to understand sentence structure
  - semantic analysis which interprets the meaning of words based on that structure.
- Parsing: parsing is the process of analyzing the grammatical structure of a sentence to understand how the words relate to each other.

Can be of two types:

- Dependency Parsing: Identifies relationships between words (e.g., subject, verb, object).
- Constituency Parsing: Builds syntax trees that represent the hierarchical structure of the sentence.



NP : Noun phrase

VP: Verb Phrase

PP: Prepositional Phrase

- These techniques support applications like machine translation and speech recognition.
- Additionally, self-supervised learning (SSL) plays a crucial role in NLP by reducing the need for manually labeled data, making model training more scalable and cost-effective.

### 1. Rule Based NLP:

- The earliest NLP applications were simple if-then decision trees, requiring preprogrammed rules
- Rule-Based Natural Language Processing uses manually defined linguistic rules to analyze and process text.
- These rules are based on grammar, syntax, and predefined patterns created by experts.
- Key Features:
  - Uses if-then rules, regular expressions, and dictionaries.
  - Effective for structured tasks like tokenization, POS tagging, and simple entity recognition.
  - Does not require large datasets for training.
- Limitations:
  - Struggles with ambiguity and language variation
  - Hard to scale and maintain for complex or diverse language inputs.
  - Because there is no machine learning or AI capability in rules-based NLP, this function is highly limited and not scalable.
- Example: A rule like if a word ends with "ed", label it as a past-tense verb.

### 2. Statistical NLP:

- Statistical Natural Language Processing uses mathematical models to understand and generate human language.
- It relies on machine learning to automatically extract, label, and classify parts of text or speech, then assigns probabilities to possible meanings.
- Instead of hand-coded rules, it uses statistics to learn patterns for example, how likely a word follows another (like in Markov models or regression models).
  - Markov Models: predict the next word in a sentence based on the previous word(s)
    - Eg if we have "I am going to" ;  
A Markov model might say : Probability of "school" after "to" = 0.6;  
Probability of "play" after "to" = 0.3;  
Probability of "run" after "to" = 0.1;  
So, it predicts "school" is most likely to come next.
  - Regression Models: used to predict a numerical outcome based on one or more input features

- This approach enabled early tools like spell checkers, T9 predictive texting, and part-of-speech tagging.

### 3. Deep Learning NLP:

- Deep learning NLP uses neural networks to process large amounts of unstructured text and voice data with high accuracy.
- It builds on statistical NLP but goes further by learning deeper language patterns automatically from data.
- It is now the dominant approach in modern NLP systems.

There are several sub-categories of models:

- Sequence-to-Sequence (Seq2Seq) Models:
  - Use RNNs to translate text from one language to another by mapping input sequences to output sequences.
- Transformer Models:
  - Use self-attention and token positions to understand relationships between words; BERT is a key example used in Google Search.
- Autoregressive Models:
  - Predict the next word in a sequence; examples include GPT, LLaMA, Claude, and Mistral, used for advanced text generation.
- Foundation Models:
  - Large pre-trained models (e.g., IBM Granite) used across tasks like content creation, entity recognition, and linking to external knowledge sources (e.g., RAG).

### How NLP Works:

#### 1. Text Pre-processing:

- Tokenization : splitting the text into smaller units like words, sentences or phrases
- Stemming or Lemmatization: reduces words to their root form (eg. running becomes run) making it easier to analyze language
- text cleaning

#### 2. Feature Extraction : Converting raw text into numerical representation

- Bag-of-Words: Each document is converted into a vector of word counts
- TF-IDF:
  - Term Frequency (TF): Measures how often a word appears in a single document.
  - Inverse Document Frequency (IDF): Gives higher weight to words that are rare across many documents, and lowers weight for very common words like "the" or "and".
- Word-embeddings: embeddings capture semantic similarity , words with similar meanings have vectors close together. For example, "king" and "queen" will have similar vectors,

#### 3. Text Analysis:

- part-of-speech (POS) tagging
- Name Entity Recognition (NER)
- Dependency parsing : relationships between words (grammatical structure)
- sentiment analysis
- NLU (extracting meaning from sentences)

4. Model Training: Processed data is then used to train machine learning models, which learn patterns and relationships within the data.

### Key Challenges in NLP:

- Ambiguity:
  - Words often have multiple meanings depending on context.
  - Example: "bank" can mean a financial institution or the side of a river.

- **Synonymy and Polysemy:**
  - Different words can mean the same thing (synonymy), and the same word can have multiple meanings (polysemy).
  - Example: “big” and “large” are synonyms; “bat” can mean an animal or a sports tool.
- **Contextual Understanding:**
  - Sentence meaning often depends on prior text or real-world knowledge.
  - Example: "He went to the bank" — the meaning of "bank" depends on previous sentences.
- **Sarcasm and Irony:**
  - Detecting sarcasm or irony is difficult because literal meaning and actual intent differ.
  - Example: “Great job!” can mean the opposite if said sarcastically.
- **Idioms and Figurative Language:**
  - These phrases cannot be interpreted literally.
  - Example: “It’s raining cats and dogs” means it’s raining heavily.
- **Variations in Language:**
  - Accents, dialects, slang, and errors add to the complexity.
  - Example: “Wanna” vs. “want to”, or regional slang like “y’all”.

## 6.1 Language Models:

- At a core of a NLP, a language model is a system that learns the patterns, grammar, and semantics of a language from a vast amount of text data.
- Its primary goal is to assign a probability to a sequence of words, indicating how likely that sequence is to occur in a natural language.
- For instance, a language model would assign a higher probability to "the cat sat on the mat" than to "mat on sat cat the."
- This capability is fundamental to many NLP applications, as it allows machines to:
  - Predict the next word (e.g., in predictive text or autocomplete).
  - Generate human-like text (e.g., for chatbots, content creation).
  - Understand context and meaning (e.g., for translation, sentiment analysis).

- **Formal Definition:**

Given a sequence of words  $w_1, w_2, \dots, w_n$ , a language model estimates:

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i | w_1, \dots, w_{i-1})$$

which is:

$$P(w_1, w_2, \dots, w_n) = P(w_1) \cdot P(w_2 | w_1) \cdot P(w_3 | w_1, w_2) \cdot \dots \cdot P(w_n | w_1, \dots, w_{n-1})$$

- Example: “I Love Pizza”

Lets break it down into equation

$$P(\text{"I love pizza"}) = P(\text{"I"}) \cdot P(\text{"love"} | \text{"I"}) \cdot P(\text{"pizza"} | \text{"I love"})$$

Lets say (example numbers):

$$P(\text{"I"}) = 0.1$$

$$P(\text{"Love"} | \text{"I"}) = 0.5$$

$$P(\text{"PIzza"} | \text{"I Love"}) = 0.6$$

Then,

$$P(\text{"I love Pizza"}) = 0.1 * 0.5 * 0.6 = 0.03$$

This is the probability of the full sentence.

### Purpose of Language Models:

- Text Generation: Predict the next word(s).

- Speech Recognition: Interpret spoken words by choosing most probable word sequences.
- Machine Translation: Help translate by modeling sentence fluency.
- Spelling/Grammar Correction: Suggest corrections based on likelihood of correct form.
- Information Retrieval: Rank documents by language relevance.

### Types of Language Models:

- Statistical Methods(N-Grams)
  - N-grams
  - exponential models
  - skip-gram models
- Neural Models

### A. Statistical Models:

- Statistical Language Modeling, or Language Modeling, is the development of probabilistic models that can predict the next word in the sequence given the words that precede.
- Examples such as N-gram language modeling.

#### 1. N-grams:

- An N-gram is a contiguous sequence of n items (typically words or characters) from a given text or speech.
- N-grams are a fundamental concept in statistical language modeling used to predict the next word in a sequence.
- For a sequence of words  $w_1, w_2, \dots, w_n$ , an N-gram language model approximates the probability of a word based on the previous  $n-1$  words.
  - $N = 1$ : Unigram
  - $N = 2$ : Bigram
  - $N = 3$ : Trigram
- And so on...
- These models make assumptions about the dependency of a word only on a fixed number of previous words.
- N-gram models are the foundation of many classic NLP systems and are still used today in baseline models, especially when computational resources are limited or interpretability is required.

#### a. Unigram Model ( $N=1$ ):

- Assumes each word is independent of the others.
- Simplest model; treats every word as occurring independently.

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i)$$

- Example: Sentence: "I love NLP"
  - Unigrams: [I], [love], [NLP]
- Pros: Very simple; Requires minimal data
- Cons: Ignores word order and context

#### b. Bigram Model ( $N=2$ )

- Assumes that the probability of a word depends only on the previous word.

$$P(w_1, w_2, \dots, w_n) = P(w_1) \cdot \prod_{i=2}^n P(w_i | w_{i-1})$$

- Example: Sentence: "I love NLP"
- Bigrams: [I love], [love NLP]
- Pros: Captures some context and word order; More accurate than unigram
- Cons: Still limited to very short context

c. Trigram Model (N = 3)

- Each word is predicted based on the two preceding words.

$$P(w_1, w_2, \dots, w_n) = P(w_1) \cdot P(w_2|w_1) \cdot \prod_{i=3}^n P(w_i|w_{i-2}, w_{i-1})$$

- Example - Sentence: "I love NLP techniques"
- Trigrams: [I love NLP], [love NLP techniques]
- Pros:
  - Captures more context than unigram or bigram
  - Useful for better language modeling
- Cons:
  - Requires much more data
  - Higher risk of data sparsity

**Example:** Bigram – probability calculation:

Let's say we have this training corpus:

I love NLP  
I love AI  
I hate bugs  
You love NLP

For a sentence like : "I love NLP"

We approximate the probability of the sentence as:

$$P(\text{"I love NLP"}) = P(\text{"I"}) \cdot P(\text{"love"}|\text{"I"}) \cdot P(\text{"NLP"}|\text{"love"})$$

(Usually, a special start token <s> is used to model the probability of the first word:

$$P(\text{"I"}|\text{"<s>"})$$

The bigram probability is estimated from a training corpus using:

$$P(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})}$$

Now Calculate:

$$P(\text{"love"} | \text{"I"})$$

$$= \frac{\text{Count}(\text{"I love"})}{\text{Count}(\text{"I"})} = \frac{2}{3}$$

$$P(\text{"NLP"} | \text{"love"})$$

$$= \frac{\text{Count}(\text{"love NLP"})}{\text{Count}(\text{"love"})} = \frac{2}{3}$$

Now.

Final Sequence probability

Assuming a start token <s> and estimating  $P(\text{"I"}|\text{"<s>"})$  as 1 (or from training data if known), then:

$$\begin{aligned} P(\text{"I love NLP"}) &= P(\text{"I"}|\text{"<s>"}) \cdot P(\text{"love"}|\text{"I"}) \cdot P(\text{"NLP"}|\text{"love"}) \\ &= 1 * \frac{2}{3} * \frac{2}{3} = \frac{4}{9} \approx 0.444 \end{aligned}$$

Limitations of N-gram Models:

- **Data Sparsity:** Data sparsity refers to the problem where many possible N-grams (word sequences) do not appear in the training corpus, even though they are valid or likely to appear in real-world language.  
As a result, the model assigns zero probability to these unseen but potentially meaningful sequences.

Example: "I love machine learning."

"You love deep learning."

Now, consider the trigram:

"I love deep"

This sequence does not exist in your training data.

The trigram model will assign:

$P(\text{"deep"}|\text{"I love"})=0$ ; even though it's a perfectly valid phrase.

- **Storage and Efficiency:** Needs to store huge counts for large  $n$ .
- **Context Limitation:** Only captures local context (limited to  $n-1$  words).
- **No understanding of meaning or grammar.**

## 2. Exponential Models:

- Exponential models such as the Maximum Entropy (MaxEnt) model offer a more flexible alternative to traditional n-gram models.
- Instead of relying solely on fixed word sequences, these models can incorporate a wide variety of linguistic and contextual features (e.g., previous words, word shapes, part-of-speech tags, etc.).
- They work by assigning weights to these features and using an exponential function to combine them into a probability distribution.
- The model selects the probability distribution with maximum entropy (i.e., the most uniform, least biased distribution) that still satisfies the constraints imposed by the training data.

## 3. Skip-gram models:

- The Skip-gram model is a neural-based approach used in word embedding techniques, especially in Word2Vec.
- It aims to predict surrounding context words given a target word, within a specified window size (e.g.,  $\pm 2$  words).
- By training on large corpora, the model learns to place semantically similar words closer in the vector space.
- Skip-gram is particularly effective in capturing semantic relationships, such as analogies (e.g., king - man + woman  $\approx$  queen).

## **B. Neural Models:**

- **Neural Language Models** are a type of language model that use neural networks to learn the probability distribution over sequences of words.
- They represent words as dense vectors (called embeddings) and learn to predict the next word based on its context in a sentence.
- **Word embeddings:**
  - Each word is represented as a dense, real-valued vector.
  - Captures semantic relationships, e.g., king - man + woman  $\approx$  queen
- **Contextual Understanding:**
  - Models like LSTM and Transformers can use entire sentences or paragraphs as context.
  - Helps understand grammar, meaning, and relationships in text

- Why Neural Language Models?
  - Traditional models like N-grams have several limitations:
    - Fixed-size context (only previous 1–2 words)
    - Data sparsity
    - No understanding of word meaning or similarity
- Neural models overcome these by:
  - Learning word embeddings
  - Capturing long-range dependencies
  - Generalizing better to unseen text
- How Do They Work?
  - The model typically:
    - Converts each word into a vector embedding
    - Passes these vectors through a neural architecture:
      - Feedforward Neural Network
      - Recurrent Neural Network (RNN)
      - LSTM / GRU
      - Transformer (e.g., BERT, GPT)
    - Outputs a probability distribution over the vocabulary for the next word.

## 6.2 Part-of-Speech (POS) Tagging:

- Part-of-Speech (PoS) tagging is a fundamental NLP task that assigns grammatical categories (like noun, verb, adjective, etc.) to each word in a sentence.
- By adding syntactic and semantic information, it helps in understanding sentence structure and meaning.
- PoS tagging is crucial for applications such as machine translation, sentiment analysis, named entity recognition, and information extraction.
- It also aids in resolving word ambiguity and serves as a bridge between human language and machine understanding in complex NLP systems.
- Common TagSets Penn - Treebank Tagset (most widely used):
  - NN: Noun, singular
  - NNS: Noun, plural
  - VB: Verb, base form
  - VBZ : verb in present tense (verb+s)
  - VBD: Verb, past tense
  - VBG: Verb, gerund/present participle
  - JJ: Adjective
  - RB: Adverb
  - DT: Determiner
  - IN: Preposition
- Example: sentence = "The quick brown fox jumps over the lazy dog"
 

Output:

The → DT  
 quick → JJ  
 brown → NN  
 fox → NN  
 jumps → VBZ  
 over → IN  
 the → DT  
 lazy → JJ  
 dog → NN

- By offering insights into the grammatical structure, this tagging helps machines in understanding not just individual words but also the connections between them inside a phrase.  
For many NLP applications like text summarization, sentiment analysis, this kind of data is essential.

### POS Tagging Approaches:

#### 1. Rule-Based Tagging:

- Rule-based POS tagging assigns grammatical tags to words using a predefined set of rules, as opposed to machine learning-based methods that require training on annotated corpora.
- These rules are crafted based on morphological features (like word endings) and syntactic context, making the approach highly interpretable and transparent.
- Example: a rule might specify that words ending in “-tion” or “-ment” should be tagged as nouns, based on common suffix patterns found in English.
  - Rule: Assign the POS tag "Noun" to words ending in -tion or -ment.
  - Text: "The presentation highlighted the key achievements of the project's development."
  - Tagged Output:
    - "The" : Determiner (DET)
    - "presentation" : Noun (N)
    - "highlighted" : Verb (V)
    - "the" : Determiner (DET)
    - "key" : Adjective (ADJ)
    - "achievements" : Noun (N)
    - "of" : Preposition (PREP)
    - "the" : Determiner (DET)
    - "project's" : Noun (N)
    - "development" : Noun (N)

#### 2. Transformation Based Tagging:

- Transformation-Based Tagging (TBT) is a method for refining POS tags through a series of context-based transformations.
- Unlike statistical taggers that rely on probabilities or rule-based taggers that apply static rules, TBT starts with initial tags and improves them iteratively by applying transformation rules.
- Example: a rule might state: “Change a pronoun to a possessive adjective if it's followed by a noun in an action context.”
  - Text: "*I saw her duck.*"
  - Initial Tags: "I" → Pronoun; "saw" → Verb; "her" → Pronoun (usual guess); "duck" → Noun (usual guess)
  - Transformation Rule Applied: This rule changes "her" from Pronoun to Possessive Adjective.
  - Updated Tags: "I" → Pronoun; "saw" → Verb; "her" → poss-adv; "duck" → Noun

#### 3. Statistical POS Tagging:

- Statistical POS tagging is a computational linguistics approach that uses probabilistic models to assign grammatical categories (e.g., noun, verb, adjective) to words in a text.
- These models estimate the probability of a tag given a word and its context, enabling them to resolve linguistic ambiguities and adapt to complex grammatical structures.
- Assigns the tag with the highest probability given the context.
- Uses probabilistic models like:
  - Hidden Markov Models (HMMs)

- A model that guesses hidden labels (like parts of speech) based on visible words, assuming each label depends only on the previous label
- Conditional Random Fields (CRFs)
  - A model that guesses labels for a whole sequence (like POS tags or named entities) by looking at the entire sentence at once, not just one word at a time.

#### 4. Machine Learning-Based Tagging:

- Supervised learning models like Decision Trees, SVMs, etc.
- Learns from labeled datasets to predict tags.

#### 5. Neural Network-based Tagging:

- Uses deep learning models like Bi-LSTMs, Transformers (e.g., BERT).
- Learns contextual word representations.
- Achieves state-of-the-art accuracy.

#### Importance and Applications of P-O-S Tagging:

| Application              | Role of POS Tagging                                |
|--------------------------|----------------------------------------------------|
| Machine Translation      | Maintains grammatical structure during translation |
| Named Entity Recognition | Helps distinguish names from common nouns          |
| Information Extraction   | Identifies relevant nouns, actions, and properties |
| Sentiment Analysis       | Understands the tone through adjectives/adverbs    |
| Question Answering       | Interprets sentence structure to extract answers   |
| Speech Synthesis         | Affects pronunciation based on word function       |

#### Ambiguity in POS Tagging:

- Many words are ambiguous and can belong to multiple parts of speech:
- Example: Word - “flies”
  - Noun: "Flies are insects."
  - Verb: "He flies to London."
- POS taggers use context to determine the correct tag.

#### Challenges in POS Tagging:

- Ambiguity: Words with multiple meanings.
- Out-of-Vocabulary Words: Unknown or rare words.
- Context Dependency: The same word can play different roles.
- Code-switching / Mixed Languages: Common in social media text.

### 6.3 Grammar and Parsing:

- Grammar defines the rules and structure of a language, including how words are combined to form phrases and sentences.
- In Natural Language Processing (NLP), understanding grammar is essential for enabling machines to interpret, analyze, and generate human language accurately.
- While grammar defines the structure and rules of a language, parsing uses these rules to determine how words in a sentence relate to one another syntactically.; often represented as a tree.

#### **Grammar:**

- Grammar is defined as the rules for forming well-structured sentences.

Formal definition:

- In computational linguistics and formal language theory, a grammar is often defined as a 4-tuple:  
 $G=(N,\Sigma,P,S)$   
 Where,
  - $N$  = A finite set of non-terminal symbols (also called variables), which represent abstract grammatical categories like *Sentence (S)*, *Noun Phrase (NP)*, *Verb Phrase (VP)*, etc.
  - $\Sigma$  = A finite set of terminal symbols (actual words or tokens in the language). These are the vocabulary of the language.
  - $P$  = A finite set of production rules or grammar rules. Each rule maps a non-terminal to a sequence of terminals and/or non-terminals (e.g.,  $S \rightarrow NP VP$ ).
  - $S$  = A special symbol from  $N$ , known as the start symbol. It represents the whole sentence or utterance being generated or parsed.
- Example – lets define a simple grammar :  $G= (N,\Sigma,P,S)$   
 where,
  - $N=\{S, NP, VP, V, N\}$
  - $\Sigma=\{\text{eats, man, apple}\}$
  - $P=\{S \rightarrow NP VP, NP \rightarrow N, VP \rightarrow V NP, N \rightarrow \text{man} \mid \text{apple}, V \rightarrow \text{eats}\}$
  - $S = S$
 This grammar can generate sentence like “man eats apple”
- Example – of a grammar rule:  
 $S \rightarrow NP VP$   
 $NP \rightarrow DT NN$   
 $VP \rightarrow VB NP$
- This means:
  - A sentence (S) consists of a noun phrase (NP) followed by a verb phrase (VP).
  - A noun phrase can be a determiner (DT) followed by a noun (NN).
  - A verb phrase can be a verb (VB) followed by another noun phrase.

### 1. Context Free Grammar (CFG):

- A Context-Free Grammar is a set of recursive rewriting rules (or productions) used to generate patterns of strings.
- Each rule maps a non-terminal symbol to a sequence of non-terminal and/or terminal symbols.
- It is called "context-free" because the rules can be applied regardless of the surrounding symbols.
- Formal notation:
- A CFG is defined as a 4-tuple:
  - $G = (N, \Sigma, P, S)$
  - $N$ : Non-terminal symbols
  - $\Sigma$ : Terminal symbols (actual words)
  - $P$ : Production rules
  - $S$ : Start symbol
- Example: A set of rules that generate sentences by substituting symbols:  
 $S \rightarrow NP VP$   
 $NP \rightarrow \text{Det } N$

VP  $\rightarrow$  V NP

Det  $\rightarrow$  'the'

N  $\rightarrow$  'dog' | 'cat'

V  $\rightarrow$  'chased'

Derivations: Step-by-step expansion:

S

$\rightarrow$  NP VP

$\rightarrow$  Det N VP

$\rightarrow$  the dog VP

$\rightarrow$  the dog V NP

$\rightarrow$  the dog chased Det N

$\rightarrow$  the dog chased the cat

- Here, At each step:
  - A non-terminal is replaced by the right-hand side of a rule
  - The rule applies without caring what comes before or after the non-terminal
  - For example: When we replace NP  $\rightarrow$  Det N, we didn't check if NP was before or after VP, we just applied the rule.
  - That's the essence of context-free: Rules apply locally, only based on the symbol you're rewriting, not on its position or neighbors.

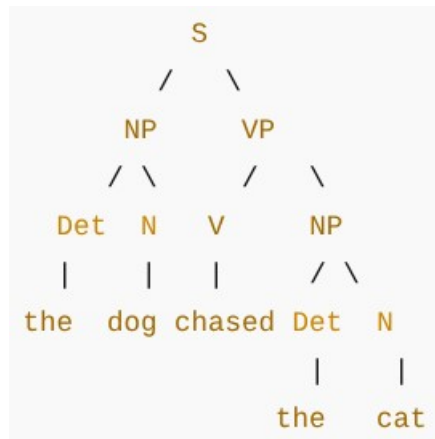
## 2. Constituent Grammar:

- Constituency grammar in NLP is a way to analyze sentence structure by breaking down into hierarchical phrases (constituents).
- These phrases, like noun phrases (NP) and verb phrases (VP), are then organized into a tree structure, showing their relationships to each other and the overall sentence.
- This contrasts with dependency grammar, which focuses on word-to-word relationships.
- CFGs are often used to define constituency grammars, using production rules to specify how constituents can be combined to form larger structures.

How it works:

- Identify Constituents:
  - These are the building blocks of a sentence, like phrases (NP, VP, etc.) and individual words
  - Examples:
    - Noun Phrase (NP): "the cat", "a large dog", "my friend"
    - Verb Phrase (VP): "sat on the mat", "is running quickly", "has eaten the fish"
    - Prepositional Phrase (PP): "on the mat", "in the park", "from Chicago"
- Build the Tree:
  - Constituency grammar represents the relationships between these constituents in a tree-like structure, with the sentence at the top and the individual words at the bottom.
  - The parser uses these rules to identify and group words into constituents, creating a hierarchical tree structure
- Analyze the relationship:
  - The tree structure reveals how the different constituents relate to each other, showing which phrases modify or depend on others.

Example: Parse tree ( built using CFG defined above)

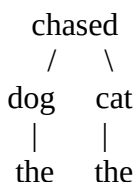


### 3. Dependency Grammar:

- Dependency grammar (DG) in natural language processing (NLP) is a way to represent sentence structure by showing how words relate to each other through dependencies.
- Unlike constituency grammars that group words into phrases, dependency grammars focus on the relationships between individual words, where one word (the head) modifies or is an argument of another (the dependent).
- These relationships are typically represented as labeled directed arcs or arrows, forming a dependency tree or graph.
- It shows which word depends on which other word.
- Main verb is the root, and other words connect to it.

Dependency Structure of “The dog chased the cat”:

- chased = main verb (root)
- dog → subject of chased
- cat → object of chased
- the → determiner for dog and cat



CFG produces constituency parse trees.

So:

- CFG ⇒ used to build ⇒ parse trees
- Parse trees ⇒ represent ⇒ constituency structure
- Dependency grammar does not use CFG or parse trees : it uses dependency trees instead

### **Parsing:**

- In natural language processing (NLP), parsing is the process of analyzing a string of text (a sentence, for example) to determine its grammatical structure.
- This involves breaking down the text into its component parts and identifying the relationships between those parts, often represented in a tree-like structure called a parse tree.
- Parsing helps computers understand the meaning of human language by revealing how words combine to form phrases and sentences.

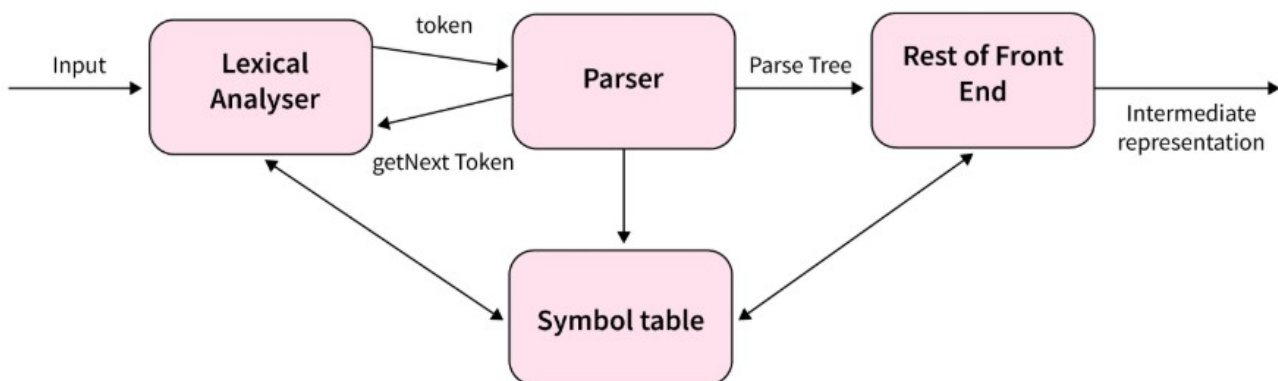
- Parsing plays a crucial role in Natural Language Processing (NLP) to analyze and understand the syntactic structure of text.
- Example: The sentence "I ate a glass of water" is grammatically incorrect in terms of meaning and should be rejected by a parser.
- NLP enables computers to interpret human language, which involves complex rules and structures beyond just identifying words.

### Why parsing is important?

- Understanding sentence structure :
  - Parsing reveals how words are combined to form phrases and clauses, which is crucial for understanding sentence meaning.
  - Applications: Machine Translation, Question Answering, Sentiment Analysis , Text summarization, Information extraction, Chatbots.
- Error Detection:
  - Parsing can help identify grammatical errors in text, improving the quality of automated language processing.

### Typical parsing process flow:

- Lexical Analysis:
  - The input text is broken down into individual tokens such as words, punctuation, or meaningful units.
  - Tool: A lexical analyzer (also called a tokenizer).
  - Example:
    - Input: "I ate a glass of water"
    - Output Tokens: ["I", "ate", "a", "glass", "of", "water"]
  - P-O-S tagging happens immediately after lexical analysis
- Symbol Table Lookup:
  - A symbol table is referenced to store and retrieve information about each token, such as:
    - Its type (noun, verb, etc.),
    - Its position in the sentence,
    - Whether it has been seen before.
  - Why it matters: This helps the parser track word meanings and roles efficiently throughout the parsing process.
- Parsing (Syntactic Analysis):
  - The tokens are fed into the parser, which uses grammar rules to analyze their syntactic structure (check is grammatically valid).
  - Output: A structured representation (tree or graph) showing how the sentence elements relate.



Example: grammar rules (CFG)

$S \rightarrow NP VP$

$NP \rightarrow Det N$

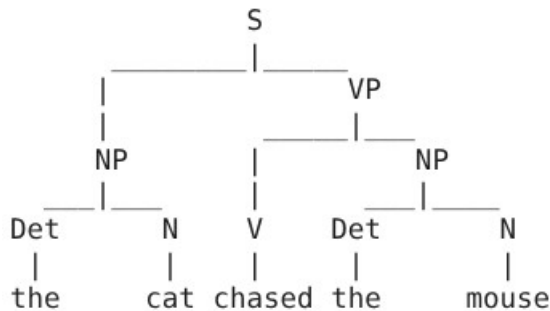
$VP \rightarrow V NP$

$Det \rightarrow \text{"the"}$

$N \rightarrow \text{"cat"} \mid \text{"mouse"}$

$V \rightarrow \text{"chased"}$

The parse tree for : “The cat chased the mouse.”



Parsing Techniques:

### 1. Top-Down Parsing:

- Top-down parsing is a parsing technique that starts from the start symbol (usually S) and tries to rewrite it into the input string by recursively applying grammar production rules, expanding non-terminals until it matches the actual sentence (tokens).

Example:

Consider the grammar:

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$VP \rightarrow V NP$

$Det \rightarrow \text{'the'}$

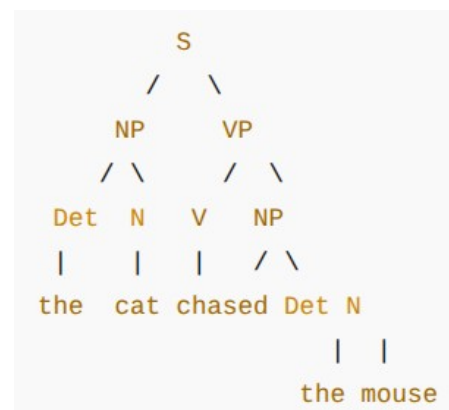
$N \rightarrow \text{'cat'} \mid \text{'mouse'}$

$V \rightarrow \text{'chased'}$

And the input sentence: “the cat chased the mouse”

Top-down parsing steps:

1. S
  2.  $\rightarrow NP VP$
  3.  $\rightarrow Det N VP$
  4.  $\rightarrow \text{'the'} N VP$
  5.  $\rightarrow \text{'the'} \text{'cat'} VP$
  6.  $\rightarrow \text{'the'} \text{'cat'} V NP$
  7.  $\rightarrow \text{'the'} \text{'cat'} \text{'chased'} NP$
  8.  $\rightarrow \text{'the'} \text{'cat'} \text{'chased'} Det N$
  9.  $\rightarrow \text{'the'} \text{'cat'} \text{'chased'} \text{'the'} N$
  10.  $\rightarrow \text{'the'} \text{'cat'} \text{'chased'} \text{'the'} \text{'mouse'}$
- SUCCESS – input fully matched



#### 1.1 Top-down parsing with backtracking:

- The parser can scan the input text repeatedly, i.e., if the input string cannot be obtained using one production rule, the parser can use another one to get the required string.
- For  $S \rightarrow A \mid B$ , if A doesn't match the input, the parser tries B.

#### 1.2 Top-down parsing without backtracking:

- Once a production rule is applied, we cannot use another one. There are two types of parsers here: Recursive Descent parser and Predictive Parser.
- Recursive Descent Parsing:

- It is a top-down parsing method that may or may not require backtracking.
- We recursively scan the input to make the parse tree, which is created from the top, while reading the input from left to right.
- Can support backtracking (if allowed), but without backtracking, each function follows one unique rule path
- Predictive Parsing:
  - It is a top-down, recursive descent parsing method that requires no backtracking.
  - A predictive parser parses the input and generates the parse tree using a stack and a parsing table.
  - To avoid backtracking, it uses a look-ahead pointer that points to the next input symbols and uses only the LL(-1) grammar (Left-to-right scanning, Leftmost derivation, 1-token lookahead).

| Step | Stack    | Input                    | Lookahead | Action                                                      |
|------|----------|--------------------------|-----------|-------------------------------------------------------------|
| 1    | S        | the cat chased the mouse | the       | Apply $S \rightarrow NP \ VP \rightarrow$ push VP, then NP  |
| 2    | VP NP    | the cat chased the mouse | the       | Apply $NP \rightarrow Det \ N \rightarrow$ push N, then Det |
| 3    | VP N Det | the cat chased the mouse | the       | Match terminal Det $\rightarrow$ the, pop Det               |
| 4    | VP N     | cat chased the mouse     | cat       | Match terminal N $\rightarrow$ cat, pop N                   |
| 5    | VP       | chased the mouse         | chased    | Apply $VP \rightarrow V \ NP \rightarrow$ push NP, then V   |
| 6    | NP V     | chased the mouse         | chased    | Match terminal V $\rightarrow$ chased, pop V                |
| 7    | NP       | the mouse                | the       | Apply $NP \rightarrow Det \ N \rightarrow$ push N, then Det |
| 8    | N Det    | the mouse                | the       | Match terminal Det $\rightarrow$ the, pop Det               |
| 9    | N        | mouse                    | mouse     | Match terminal N $\rightarrow$ mouse, pop N                 |
| 10   | (empty)  | (empty)                  | —         | Parsing successful : input and stack empty                  |

## 2. Bottom-Up Parsing:

- Bottom-up parsing is a parsing strategy that starts from the input tokens (leaves) and attempts to construct the parse tree up to the start symbol by reducing smaller parts of the sentence into higher-level grammar symbols.
- It's like assembling puzzle pieces from the bottom until you complete the whole picture (i.e., the start symbol S).

Same example:

$S \rightarrow NP \ VP$   
 $NP \rightarrow Det \ N$   
 $VP \rightarrow V \ NP$   
 $Det \rightarrow 'the'$   
 $N \rightarrow 'cat' \mid 'mouse'$   
 $V \rightarrow 'chased'$

input: “the cat chased the mouse”

Derivation in reverse : we reduce step-by-step until we reach S.

Input: the cat chased the mouse

1. the → Det
  2. cat → N
  3. Det N → NP
  4. chased → V
  5. the → Det
  6. mouse → N
  7. Det N → NP
  8. V NP → VP
  9. NP VP → S
- SUCCESS — start symbol derived

### Other types (Assignment):

Constituency parsing

Dependency Parsing

## 6.4 Complications of Real Natural Language:

Natural Language Processing (NLP) is an AI technique that enables machines to understand and generate human language.

Despite its vast potential, NLP faces several challenges due to the complexity of natural languages:

### Major Challenges in NLP:

#### 1. Language Diversity and Ambiguity:

- Human language is highly diverse and intricate, with thousands of languages, each having its own grammar, vocabulary, and cultural context. This makes understanding all languages a challenge for both humans and machines.
- Natural languages are ambiguous: words and phrases can have multiple meanings depending on context. They also involve complex syntactic and grammatical rules (like tense, word order, agreement) which make processing difficult.

#### 2. Limited and Biased Training Data:

- Requires large, annotated, and unbiased datasets.
- Bias in training data can lead to unfair models.

#### 3. High Resource Demand:

- NLP projects require substantial time and resources depending on task complexity, data quality, and model design. Complex tasks like machine translation need large annotated datasets and expert input. Training models also demands powerful hardware such as GPUs or TPUs, making development resource-intensive and time-consuming

#### 4. Navigating Phrasing Ambiguities in NLP:

- Phrasing ambiguities arise when a phrase can be interpreted in multiple ways, making it difficult for NLP systems to understand the correct meaning. Resolving this requires multiple analysis strategies:
  - Contextual Understanding: Uses surrounding text or dialogue to clarify meaning.
  - Semantic Analysis: Examines word meanings and roles using techniques like word sense disambiguation.
  - Syntactic Analysis: Analyzes grammatical structure to determine valid interpretations.
  - Pragmatic Analysis: Considers speaker intent and conversational context.
  - Statistical Methods: Applies ML models to learn patterns and predict intended meanings.

#### 5. Spelling and Grammar Errors:

- Noisy inputs affect model performance.
- Key points for solving misspelling and grammatical error in NLP:
  - Spell Checking: Uses algorithms and dictionaries to detect and correct misspelled words.
  - Text Normalization: Converts text to a standard format by lowercasing, removing punctuation, and expanding contractions.
  - Tokenization: Splits text into tokens to isolate and identify errors for easier correction.
  - Language Models: Predicts correct words or phrases based on context using trained models.

#### 6. Mitigating Innate Biases in NLP algorithms:

- Ensuring fairness, equity, and inclusivity in NLP requires active bias mitigation strategies.
- This involves collecting diverse and representative data, detecting demographic biases, preprocessing data (e.g., debiasing embeddings), training models to learn fair representations, and regularly auditing and evaluating models using fairness metrics on diverse datasets.

#### 7. Words with multiple meanings:

- Polysemous : word has multiple related meanings. Eg paper
- Homonymous : word has multiple unrelated meanings. eg. bat
- This lexical ambiguity makes it difficult for NLP systems to accurately interpret or process text.
- Solutions:
  - Contextual Analysis: Use surrounding words and sentences to determine the intended meaning.
  - Word Sense Disambiguation (WSD): Apply algorithms or models to identify the correct sense of a word based on context.
  - Pre-trained Language Models: Models like BERT or GPT understand word meaning better by leveraging large contextual embeddings.
  - Semantic Networks: Use tools like WordNet to understand and differentiate between word senses.

#### 8. Addressing multilingualism:

- It is very important to address language diversity and multilingualism in Natural Language Processing to confirm that NLP systems can handle the text data in multiple languages effectively.
- Here are some key points to address language diversity and multilingualism:
  - Multilingual Corpora: Multilingual corpus consists of text data in various languages and serve as valuable resources for training NLP models and systems.
  - Cross-Lingual Transfer Learning: This is a type of techniques that is used to transfer knowledge learned from one language to another.
  - Language Identification: Design language identification models to automatically detect the language of a given text.
  - Machine Translation: Machine Translation provides the facility to communicate and inform access across language barriers and can be used as preprocessing step for multilingual NLP tasks.

#### 9. Reducing uncertainty and False Positive in NLP:

- It is very crucial task to reduce uncertainty and false positives in Natural Language Process (NLP) to improve the accuracy and reliability of the NLP models.
- Here are some key points to approach the solution:

- Probabilistic Models: Use models like Bayesian networks to estimate uncertainty and improve decision-making.
- Confidence Scores: Measure the model's certainty in its predictions to detect potentially unreliable outputs.
- Threshold Tuning: Adjust decision thresholds to balance recall and precision, helping reduce false positives.
- Ensemble Methods: Combine multiple models to increase overall reliability and reduce prediction errors.

#### 10. Facilitating Continuous Conversations with NLP:

- This involves developing real-time NLP systems that can quickly process and respond to user input with low latency (instantly).
- Key aspects include maintaining conversational context to understand user intent, tracking dialogue history, and generating relevant responses.
- Intent recognition algorithms are used to identify the user's goals and ensure coherent, seamless interaction.

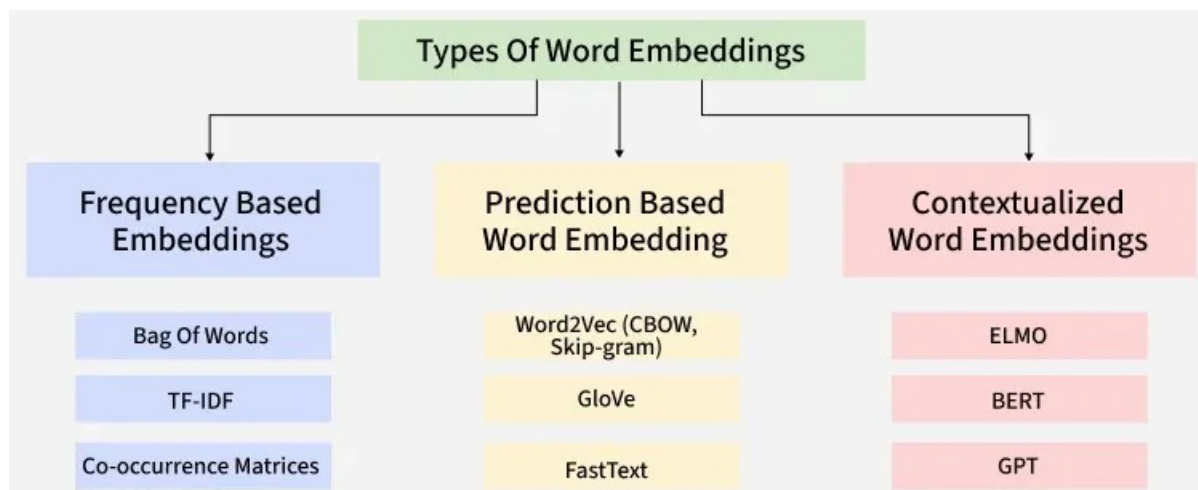
#### How to Overcome NLP Challenges:

Overcoming challenges in Natural Language Processing (NLP) requires a blend of advanced technologies, domain expertise, and strategic methods.

Below are key approaches:

- Quantity and Quality of Data:
  - Use diverse and high-quality datasets.
  - Apply techniques like data augmentation, synthesis, and crowdsourcing to overcome data scarcity
    - augmentation: modifying existing data – synonym replacement, paraphrasing
    - synthesis: artificially generate new data
    - crowdsourcing: collect data, labels or inputs from a large group of online contributors
- Handling Ambiguity: Train NLP models to disambiguate words and phrases using contextual and semantic understanding
  - contextual understanding: Use models like BERT, GPT, or RoBERTa that understand the meaning of a word based on its surrounding context. Eg. "He went to the **bank** to withdraw money" vs. "She sat on the **bank** of the river."
  - semantic understanding: Understand the actual meaning or concept behind words and phrases, not just based on context but also their roles and relationships.
- Out-of-vocabulary(OOV) words: Address OOV issues with methods like tokenization, character-level modeling, and vocabulary expansion.
  - OOV: words not seen during training, so the model doesn't recognize them.
  - Tokenization: eg. *unbelievable* → ["un", "believe", "able"]
- Lack of Annotated Data: Leverage transfer learning and pre-trained models to adapt knowledge from large datasets to tasks with limited labeled data.

## 6.5 Word Embeddings:



### Approaches for Text Representation (traditional approach / frequency based embeddings):

#### 1. One-Hot Encoding:

- In this encoding scheme, each word in the vocabulary is represented as a unique vector, where the dimensionality of the vector is equal to the size of the vocabulary.
- High-dimensional vectors, Computationally expensive and Memory-intensive
- Does not capture Semantic Relationships
- Restricted to the seen training vocabulary

Example:

Vocabulary: {'mat', 'the', 'bird', 'hat', 'on', 'in', 'cat', 'tree', 'dog'}

Word to Index Mapping: {'mat': 0, 'the': 1, 'bird': 2, 'hat': 3, 'on': 4, 'in': 5, 'cat': 6, 'tree': 7, 'dog': 8}

One-Hot Encoded Matrix:

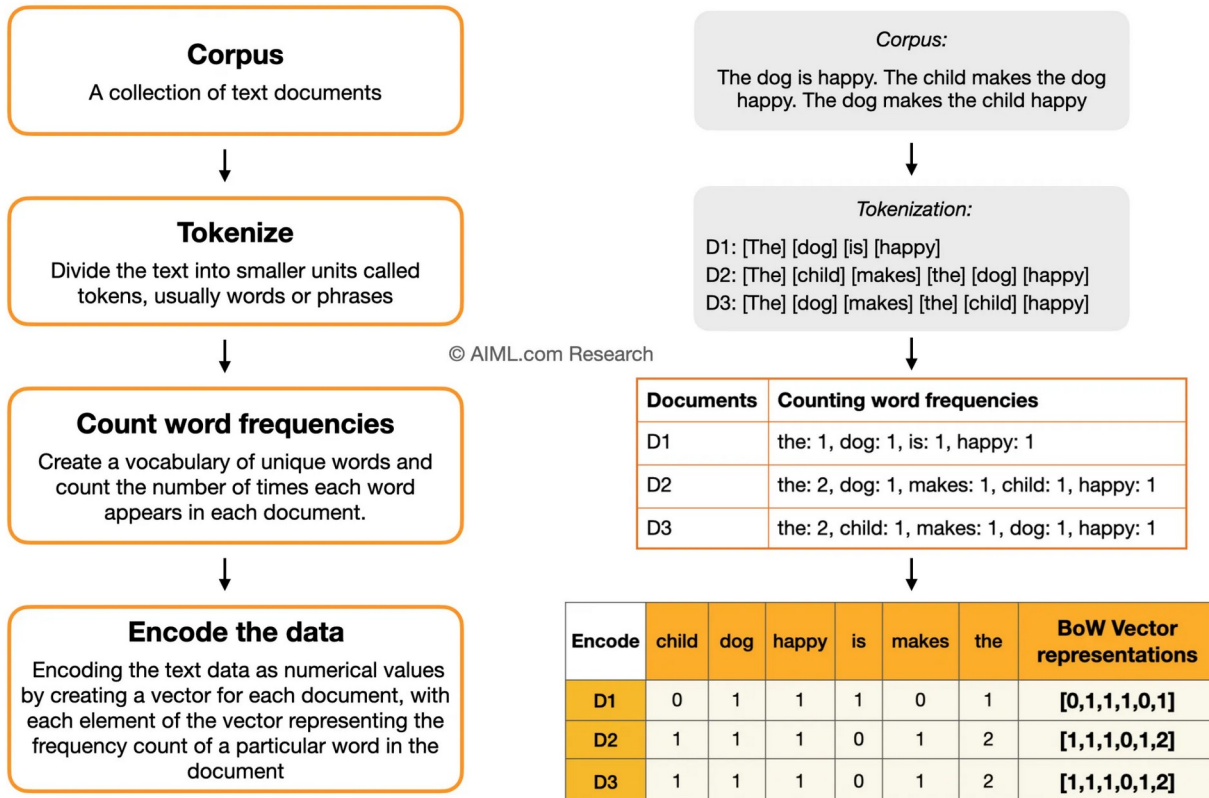
```
cat: [0, 0, 0, 0, 0, 0, 1, 0, 0]
in: [0, 0, 0, 0, 0, 1, 0, 0, 0]
the: [0, 1, 0, 0, 0, 0, 0, 0, 0]
hat: [0, 0, 0, 1, 0, 0, 0, 0, 0]
dog: [0, 0, 0, 0, 0, 0, 0, 0, 1]
on: [0, 0, 0, 0, 1, 0, 0, 0, 0]
the: [0, 1, 0, 0, 0, 0, 0, 0, 0]
mat: [1, 0, 0, 0, 0, 0, 0, 0, 0]
bird: [0, 0, 1, 0, 0, 0, 0, 0, 0]
in: [0, 0, 0, 0, 0, 1, 0, 0, 0]
the: [0, 1, 0, 0, 0, 0, 0, 0, 0]
tree: [0, 0, 0, 0, 0, 0, 0, 1, 0]
```

#### 2. Bag-of-Words:

- Bag-of-Words (BoW) is a text representation technique that represents a document as an unordered set of words and their respective frequencies.
- Ignores the order of words in the document: Causes loss of sequential information and context
- Sparse representations make it Memory intensive: Many elements are zero resulting in Computational inefficiency with large datasets.

## Bag of Words Model explanation

## Example



### 3. Term Frequency – Inverse Document Frequency (TF-IDF):

- Term Frequency-Inverse Document Frequency, commonly known as TF-IDF, is a numerical statistic that reflects the importance of a word in a document relative to a collection of documents (corpus).
- It is widely used in natural language processing and information retrieval to evaluate the significance of a term within a specific document in a larger corpus.
- TF-IDF consists of two components:
  - Term Frequency (TF): Measures how often a term (word) appears in a document.
  - Inverse Document Frequency (IDF): Measures the importance of a term across a collection of documents.
- The TF-IDF score for a term  $t$  in a document  $d$  is then given by multiplying the TF and IDF values:

$$TF - IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

where,

**Term Frequency (TF):**  $TF(t, d) = \frac{\text{Total number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$

**Inverse Document Frequency (IDF):**  $IDF(t, D) = \log \left( \frac{\text{Total documents}}{\text{Number of documents containing term } t} \right)$

- The higher the TF-IDF score for a term in a document, the more important that term is to that document within the context of the entire corpus.
- This weighting scheme helps in identifying and extracting relevant information from a large collection of documents, and it is commonly used in text mining, information retrieval, and document clustering.

Example:

- Text A: Jupiter is the largest planet.

- Text B: Mars is the fourth planet from the Sun.

The table below shows the values of TF for A and B, IDF and TFIDF for A and B.

| Words   | TF ( A ) | TF ( B ) | IDF             | TFIDF ( A ) | TFIDF ( B ) |
|---------|----------|----------|-----------------|-------------|-------------|
| jupiter | 1/5      | 0        | $\ln(2/1)=0.69$ | 0.138       | 0           |
| is      | 1/5      | 1/8      | $\ln(2/2)=0$    | 0           | 0           |
| the     | 1/5      | 2/8      | $\ln(2/2)=0$    | 0           | 0           |
| largest | 1/5      | 0        | $\ln(2/1)=0.69$ | 0.138       | 0           |
| planet  | 1/5      | 1/8      | $\ln(2/2)=0$    | 0.138       | 0           |
| mars    | 0        | 1/8      | $\ln(2/1)=0.69$ | 0           | 0.086       |
| fourth  | 0        | 1/8      | $\ln(2/1)=0.69$ | 0           | 0.086       |
| from    | 0        | 1/8      | $\ln(2/1)=0.69$ | 0           | 0.086       |
| sun     | 0        | 1/8      | $\ln(2/1)=0.69$ | 0           | 0.086       |

In vector representation:

A = [0.138 0 0 0.138 0.138 0 0 0 0]

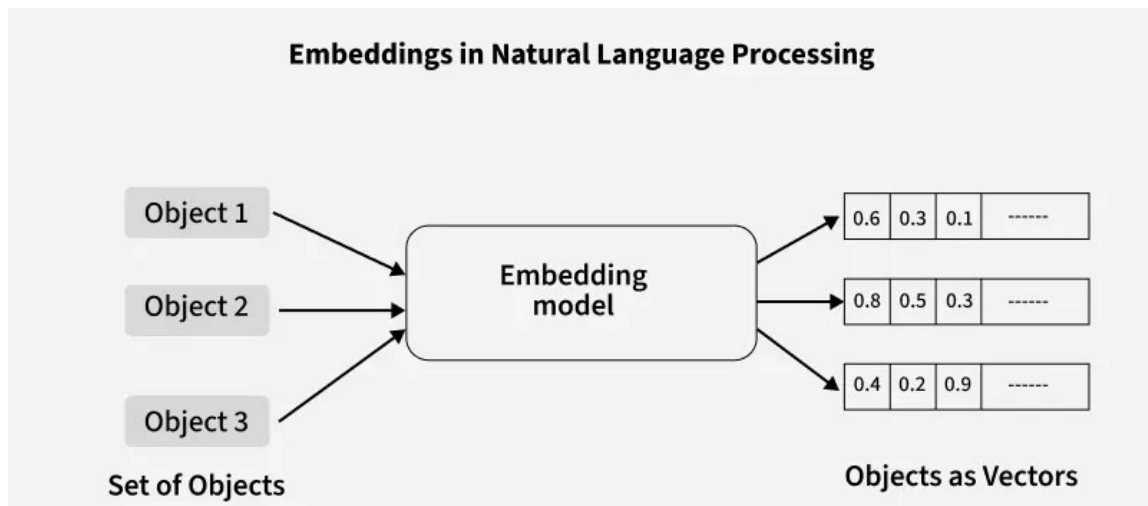
B = [0 0 0 0 0 0.086 0.086 0.086 0.086]

### Approaches for Text Representation: Word Embeddings:

- Word Embedding is a technique used to represent words and documents as numeric vectors in a lower-dimensional space.
- These word vectors serve as numerical inputs that encode the meaning of words efficiently.
- It is a method for extracting meaningful features from text, enabling these features to be fed into machine learning algorithms for processing textual data.
- Word embeddings ensure that words with related meanings have similar vector representations, which allows for measuring similarity through their vector closeness.
- One challenge with raw text representation is the high computational cost due to very large input vectors and consequently many parameters.
- Embeddings address this by significantly reducing the dimensionality while still maintaining important syntactic and semantic relationships within the data.

### Need for Word Embedding:

- Overcoming Sparse Representations:
  - traditional – one-hot vectors are high dimensional and sparse (mostly zero)
  - word embedding uses low dimensional vectors, making computation more efficient
- Capture semantic meaning: allows words with similar meanings (eg king and queen) to have similar vectors
- Improved machine learning performance
- Dimensionality Reduction : reduce computational burden by avoiding large vectors for each word
- Preserving contextual relationships: syntactic and semantic relationship between words (eg. walk → walking; run → running; relationship: verb → verb+ing form)
- Handling vocabulary efficiently : some embeddings can handle out-of-vocabulary (OOV)
- Foundation for model NLP models : lstm, cnns, transformers



Example:

- Imagine you have a vocabulary of 1000 different animals.
- Traditionally, you could represent each animal using a one-hot vector of size 1000 : where only one position is 1 (the animal), and the rest are 0s. This is sparse and has no semantic information.
- But with word embeddings, instead of using 1000 binary features, we can represent each animal using a vector of just (lets say 100 features) (i.e., a 100-dimensional vector), a dense vector.

| Feature # | king  | queen | dog   | banana | ... | zebra |
|-----------|-------|-------|-------|--------|-----|-------|
| Feature 1 | 0.221 | 0.217 | 0.102 | 0.089  | ... | 0.298 |
| Feature 2 | 0.569 | 0.565 | 0.778 | 0.124  | ... | 0.459 |
| Feature 3 | 0.338 | 0.337 | 0.135 | 0.302  | ... | 0.676 |
| ...       | ...   | ...   | ...   | ...    | ... | ...   |
| Feat. 100 | 0.951 | 0.945 | 0.423 | 0.712  | ... | 0.883 |

**Prediction-based Approaches of Word-embeddings: Word2Vec:**

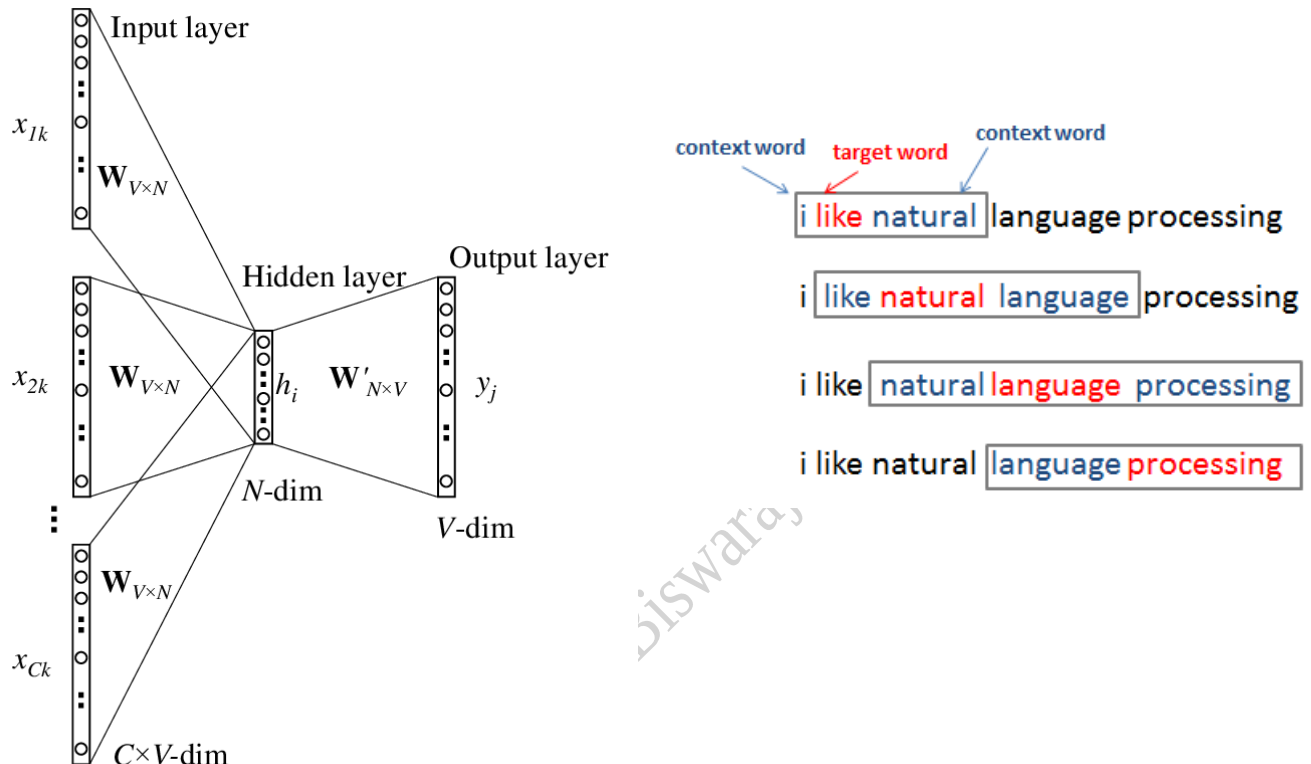
- Word2Vec is a neural approach for generating word embeddings.
- It belongs to the family of neural word embedding techniques, specifically under the category of distributed representation models.
- Word2Vec is a widely used technique in natural language processing (NLP).
- It represents words as continuous vector spaces.
- The main aim is to capture semantic relationships between words by mapping them into high-dimensional vectors.
- Words with similar meanings are expected to have similar vector representations.
- Every word is assigned a vector.
  - This typically begins with either a random vector or a one-hot encoded vector, which is then trained into a meaningful embedding through learning.

There are two neural embedding methods for Word2Vec: **Continuous Bag of Words (CBOW)** and **Skip-gram**.

1. Continuous Bag-of-Words:

- CBOW is one of the two model architectures used in Word2Vec.
- It predicts the target word based on the context words surrounding it.
- Input: Context words (surrounding words)

- Output: The central (target) word
- The model learns by adjusting word vectors to minimize prediction error.
- It works well with frequent words and is faster to train than Skip-Gram.
- Example:
  - Sentence: "The cat sat on the mat"
  - Context: [The, cat, on, the] → Target: "sat"



- Feedforward neural network with a single hidden layer.
- The input layer, hidden layer, and output layer represent the context words, learned continuous vectors or embeddings, and the target word.
- Useful for learning distributed representations of words in a continuous vector space.
- The hidden layer contains the continuous vector representations (word embeddings) of the input words.
- The weights between the input layer and the hidden layer are learned during training.
- The dimensionality of the hidden layer represents the size of the word embeddings (the continuous vector space).

## 2. Skip-Gram:

- The main objective of Skip-Gram is to predict context words (words surrounding a target word) given a target word.
- This is the opposite of the Continuous Bag of Words (CBOW) model, where the objective is to predict the target word based on its context.
- Example – Sentence: "The cat sits on the mat"
  - Suppose: Center word: "sits"
  - Window size: 2
  - Then the context words are:
    - → "The", "cat", "on", "the"
  - Training Pairs:

- ("sits", "The")
  - ("sits", "cat")
  - ("sits", "on")
  - ("sits", "the")
- This is how training data is constructed for skip-gram.
- Input Layer: Takes a one-hot encoded word (e.g., "sits") : a vector with 1 at the word's index and 0s elsewhere.
- Hidden Layer: Maps the input to a fixed-size dense vector (e.g., 100-dimensional) : this becomes the word embedding.
- Output Layer: Predicts which words are likely to appear around the input word by scoring all vocabulary words.

### Prediction-based Approaches of Word-embeddings: GloVe:

- Derive the semantic relationship between words using word-to-word co-occurrence matrix.
- GloVe is trained using global word co-occurrence statistics.
- It utilizes the overall context of the corpus to generate word embeddings that capture the meaning of words based on how frequently they appear together.
- The process involves scanning the entire corpus and recording how often each word appears alongside others. This results in a co-occurrence matrix.
- Words that appear directly next to each other are assigned a value of 1; if they are separated by one word, the value is 1/2; if two words apart, 1/3; and so on.

#### Example:

- Let's see how the matrix is created.

Corpus:

- It is a nice evening.
- Good Evening!
- Is it a nice evening?

|         | it      | is      | a       | nice | evening | good |
|---------|---------|---------|---------|------|---------|------|
| it      | 0       |         |         |      |         |      |
| is      | 1+1     | 0       |         |      |         |      |
| a       | 1/2+1   | 1+1/2   | 0       |      |         |      |
| nice    | 1/3+1/2 | 1/2+1/3 | 1+1     | 0    |         |      |
| evening | 1/4+1/3 | 1/3+1/4 | 1/2+1/2 | 1+1  | 0       |      |
| good    | 0       | 0       | 0       | 0    | 1       | 0    |

- The upper half of the matrix will be a reflection of the lower half.
- We can consider a window frame as well to calculate the co-occurrences by shifting the frame till the end of the corpus.
- This helps gather information about the context in which the word is used.
- Vectors for each word is assigned randomly.
- Take two pairs of vectors and see closeness in space.
- If they occur together more often or have a higher value in the co-occurrence matrix and are far apart in space then they are brought close to each other.
- If they are close to each other but are rarely or not frequently used together then they are moved further apart in space.

- So GloVe moves words around until this map reflects how words co-occur in real life
- Output: Vector space representation that approximates the information from the co-occurrence matrix.
  - Each word now has a position : like a coordinate. That's the word embedding

#### Limitation:

- Context Insensitivity (Static Embeddings):
  - GloVe assigns a single vector to each word, so it cannot differentiate between different meanings of the same word (polysemy).
  - For example:
  - The word "bank" in "river bank" and "financial bank" will have the same embedding, even though their meanings are different in each context.

#### **Prediction-based Approaches of Word-embeddings: Fast Text (Facebook AI):**

- FastText is an extension of the Word2Vec model developed by Facebook AI Research (FAIR) that generates word embeddings by representing words as a combination of character n-grams.
- Unlike Word2Vec or GloVe which treat each word as a single atomic unit, FastText breaks words into subword units (character n-grams) and learns embeddings for those.
- The final word embedding is obtained by averaging the embeddings of all its subwords.
- How it works?
  - A word like “playing” is broken into n-grams:
  - For n=3 (trigrams): pla, lay, ayi, yin, ing
  - The word embedding for playing is the sum or average of the embeddings of these subword units.
  - This allows the model to capture morphological information and handle out-of-vocabulary (OOV) words better.
- Training: Can use CBOW or Skip-gram architecture like Word2Vec.

#### **Contextual Word Embeddings:**

- Contextual word embeddings are word representations that change depending on the context in which a word appears.
- Traditional embeddings like Word2Vec, GloVe, or FastText give one fixed vector for each word.
- That means:
  - The word “bank” has the same vector in
  - “I sat by the river bank”
  - and “I went to the bank to deposit money”
  - Even though the meaning is very different!
- Contextual Embeddings fix this:
- They give different vectors depending on the sentence.
- In BERT or GPT models:
  - The model reads the entire sentence.
  - It uses transformers and attention to understand how words relate to each other.
  - Then it generates a unique vector for each word based on its surrounding words (context).
- Example: for the word "apple":

| Sentence                       | Meaning      | Vector will reflect... |
|--------------------------------|--------------|------------------------|
| "I ate an apple after lunch."  | fruit        | - food context         |
| "Apple released a new iPhone." | tech company | - technology context   |

- The vector for “apple” is not reused, but recomputed in each context.

## 6.6 RNN for NLP:

### Why RNN?

- Traditional neural networks like Feedforward Neural Networks (FNNs) and Convolutional Neural Networks (CNNs) are not designed to handle sequential data such as text, speech, time series, or video frames: where order and context over time matter.
- They treat each input independently and cannot remember previous inputs.
- RNNs are needed when the order, timing, and memory of previous inputs affect the current prediction like in language modeling, speech recognition, or stock prediction.

### RNN Architecture:

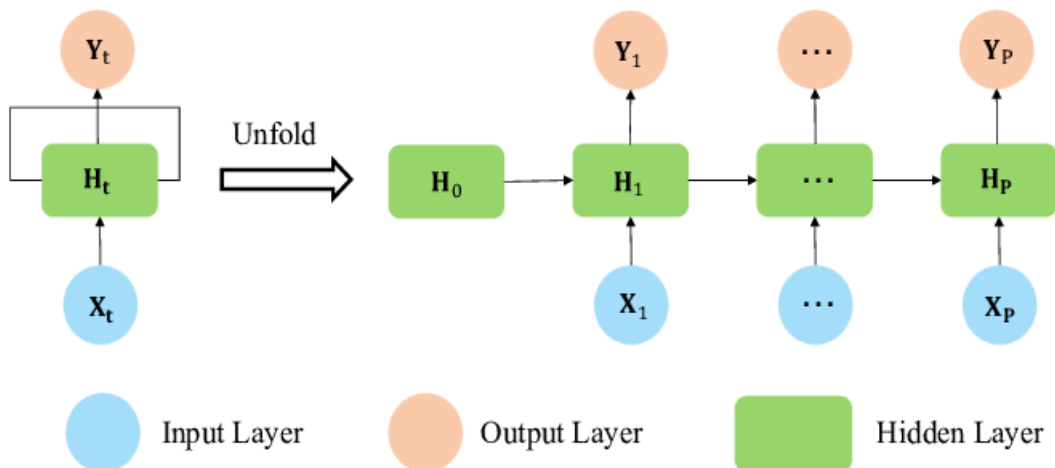


fig. RNN architecture

- A Recurrent Neural Network (RNN) is a type of neural network designed to process sequential data by using memory of past inputs through internal hidden states.
- It is especially useful when the order of data matters, such as in language, time series, or audio processing.
- The core idea of RNN architecture is that it reuses the same neural unit across different time steps, passing information forward through a hidden state.
- At each time step:
  - It takes the current input and the previous hidden state,
  - Computes the new hidden state,
  - And optionally produces an output.
- This creates a chain-like architecture, where each unit is connected to the next through time.

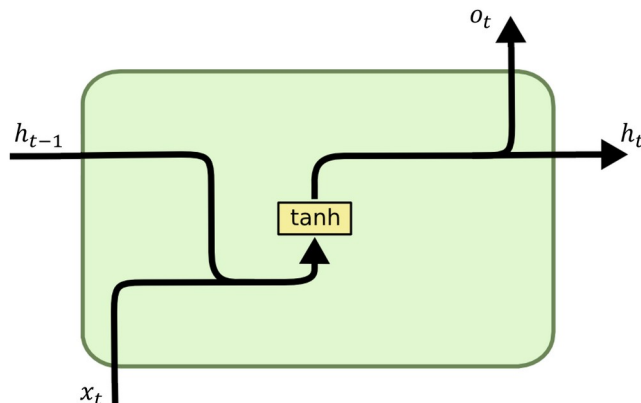


Fig. A simple RNN Cell

- At time step  $t$ :

○ **Hidden State** 
$$h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

**Output** 
$$y_t = \psi(W_{hy}h_t + b_y)$$

where,

$W_{xh}$ : weight matrix from input to hidden state

$W_{hh}$ : weight matrix from previous hidden state

$W_{hy}$ : weight matrix from hidden state to output

$b_h$ : bias for hidden layer

$b_y$ : output bias

$\phi$ : activation function (commonly ***tanh*** or ***ReLU***)

$\psi$ : output activation function (e.g., ***softmax*** for classification, ***linear*** for regression)

#### Problems with RNN: Vanishing and Exploding Gradient:

- When training RNNs, we use Backpropagation Through Time (BPTT), the gradients are propagated backward across many time steps.

$$W_{\text{new}} = W_{\text{old}} - \eta \cdot \frac{\partial \mathcal{L}}{\partial W_{\text{old}}}$$

- During this process, gradients are repeatedly multiplied by the RNN's weight matrices. Depending on the size of the weights, this can cause two major issues:

##### 1. Vanishing Gradient Problem

- When gradients become very small as they move backward in time.
- Happens when weights have magnitudes less than 1.
- Causes exponential decay of gradients over time steps.
- Why is it bad?
  - The RNN stops learning long-range dependencies.
  - Earlier time steps have little to no influence on the current output.
  - Model fails to "remember" important distant context.
- Solution: Use LSTM

##### 2. Exploding Gradient Problem

- When gradients become very large as they move backward.
- Happens when weights have magnitudes greater than 1.
- Causes exponential growth of gradients.
- Why is it bad?
  - Leads to unstable training.
  - Sudden weight updates, numerical overflow, and divergence of loss (increasing rapidly).
    - Normally we want: Loss:  $1.2 \rightarrow 0.9 \rightarrow 0.6 \rightarrow 0.3 \rightarrow 0.1$  (convergence)
    - But in case of divergence: Loss:  $1.2 \rightarrow 4.8 \rightarrow 100 \rightarrow \infty$  (divergence)
- Solution: Applying gradient clipping (limiting), reduce learning rate

#### **6.7 LSTMs for NLP:**

- Long Short-Term Memory (LSTM) is a special type of RNN designed to overcome the vanishing gradient problem and to learn long-term dependencies more effectively.
- It introduces gates and a memory cell to control the flow of information across time steps.
- Widely used in tasks like text generation, translation, speech recognition, and time series forecasting.

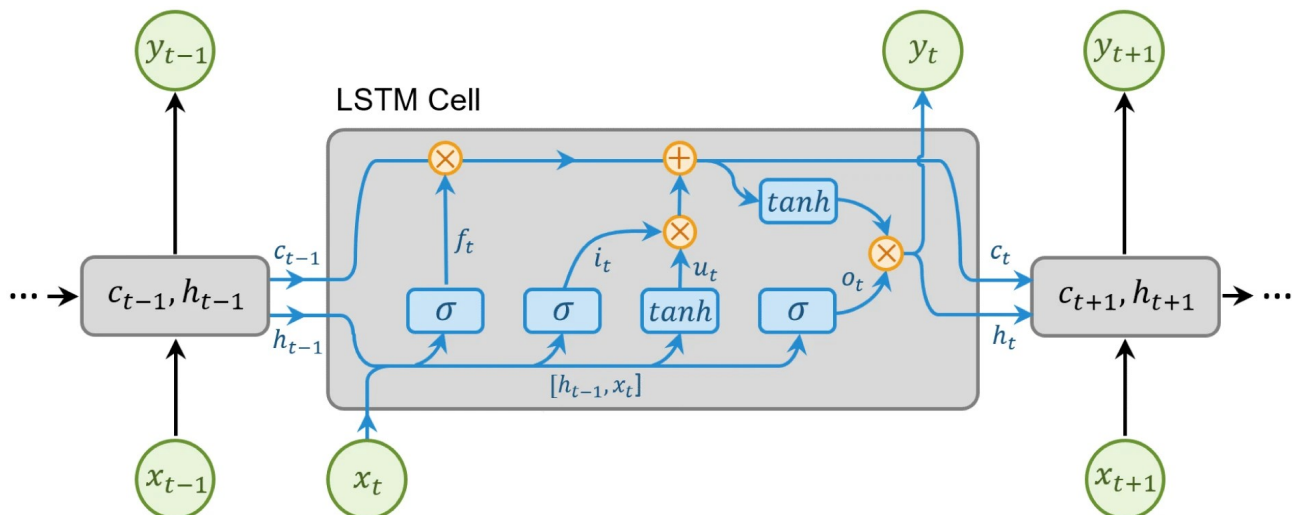
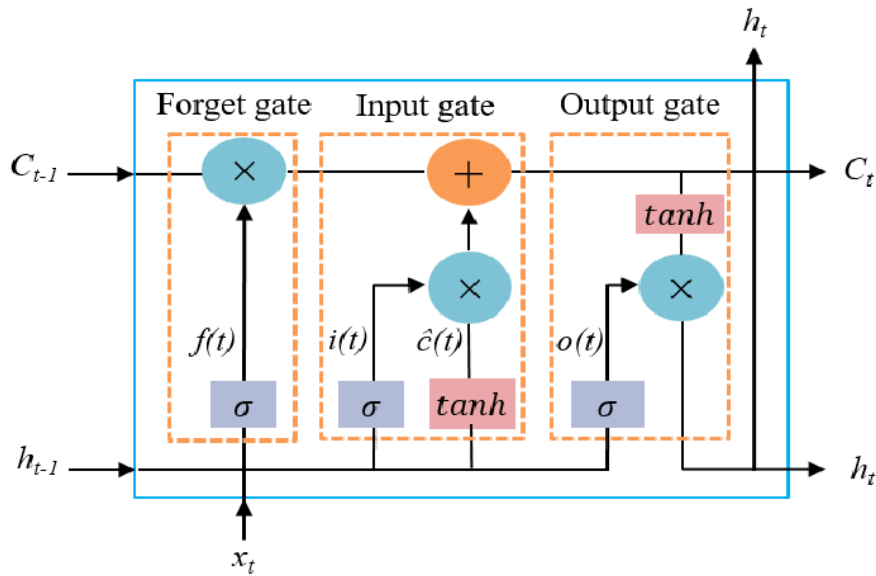


fig. architecture of LSTM

### Lstm Architecture:

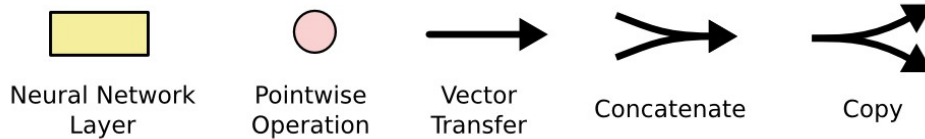
- Unlike a RNN, each LSTM cell has:
  - Cell State ( $C_t$ )
  - Hidden State ( $h_t$ )
  - Gates: Structures that decide what to keep, what to forget, and what to output.
    - Forget Gate
    - Input Gate
    - Output Gate
- LSTM's Key Innovation: The Cell State ( $C_t$ ):
  - Acts like a memory conveyor belt.
  - Long-term memory that flows through the entire sequence with minimal changes
  - Information flows almost unchanged across time steps, with only minor linear interactions
  - Applying activation functions like **tanh** or **sigmoid** compress values into a small range (e.g., -1 to 1 or 0 to 1).
  - If you keep applying these at every step, the values get weaker and weaker, and eventually vanish ( $\rightarrow$  vanishing gradient problem).
  - In the main memory path (cell state), LSTM doesn't repeatedly squash(compress) values.
  - It only uses these functions at specific gates, not on the main memory every time.
  - Since the main memory path is mostly left unchanged, gradients (used in learning) can flow backward easily during training.
- Role of Gates in LSTM:

| Gate             | Purpose                                               | Effect on Gradient Flow           |
|------------------|-------------------------------------------------------|-----------------------------------|
| Forget Gate "ft" | Decides what to erase from the cell state             | Keeps useful info, removes noise  |
| Input Gate "it"  | Decides what new information to write into cell state | Adds new memory selectively       |
| Output Gate "ot" | Decides what part of the memory to expose as output   | Controls exposure of hidden state |

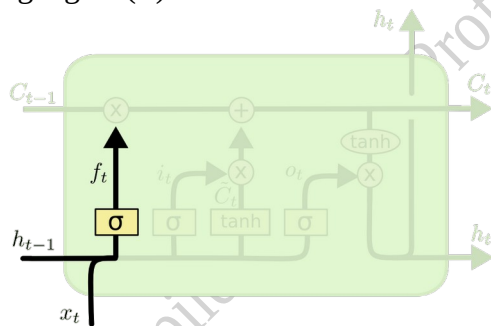


### Mathematical Equations:

- $\sigma$  : sigmoid activation

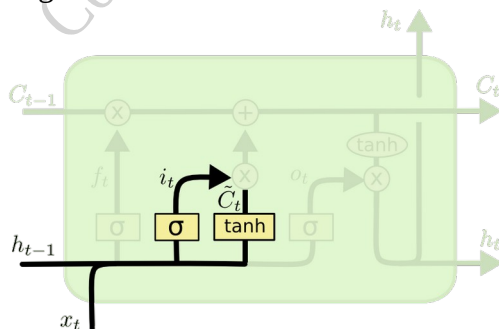


- forget gate (ft):



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- input gate:

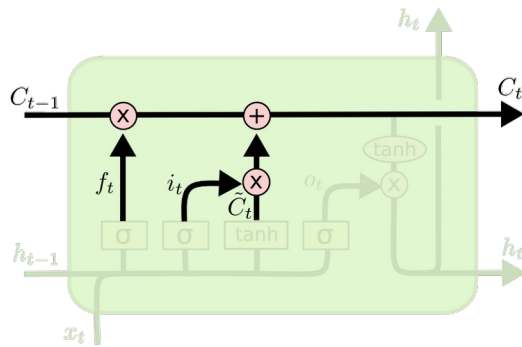


$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

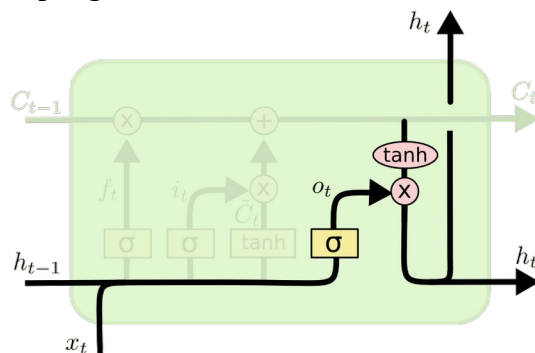
- $\tilde{C}_t$  is the candidate cell state

- Update to cell state:



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

- output gate and hidden state:



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

### Bidirectional RNN / LSTM:

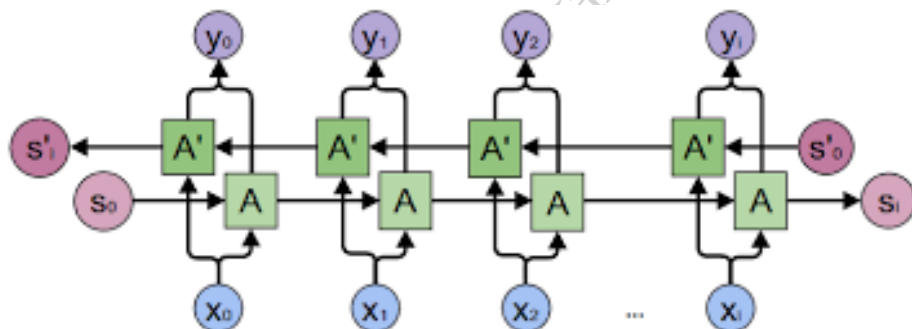


fig. Bi-lstm / rnn architecture

- A Bi-Directional LSTM is a special type of LSTM that processes the input in both forward and backward directions, and then combines both results.
- It's designed to give the model access to past and future context in a sequence.
- Why Use It?
  - Standard (unidirectional) LSTMs only look left to right:
  - Sentence: "The movie was amazing"
  - LSTM sees: The → movie → was → amazing
  - At each step, it only knows the words before the current one.
  - But sometimes, knowing future words is important too!
  - For example:
    - "He said the film was surprisingly good."
  - If you're at the word "film", it helps to know what's coming next: "was surprisingly good".
- How BiLSTM Works
  - It runs two LSTMs:
  - One from left to right (forward)

- One from right to left (backward)
  - At each time step, it concatenates or adds the two outputs:
    - $\text{output}_t = [\rightarrow \text{LSTM}_t; \leftarrow \text{LSTM}_t]$
- Benefits
  - Captures context from both directions
  - Useful for tasks like:
    - Named Entity Recognition (NER)
    - Text classification
    - Machine translation
    - POS tagging
- Limitation
  - Not suitable for real-time streaming, where future tokens aren't available.
  - Slightly more computationally expensive (double LSTM)

## 6.8 Sequence-to-Sequence (Seq2Seq) Models:

- The Sequence-to-Sequence (Seq2Seq) model is a neural network structure used to convert one sequence into another.
- It takes an input sequence, processes it, and produces a corresponding output sequence.
- This model is widely applied in machine learning tasks like natural language processing, machine translation, and speech recognition.
- Both the input and the output are treated as sequences of varying lengths and the model is composed of two parts:
  - Encoder: It processes the input sequence and encodes it into a fixed-length context vector or series of hidden states.
  - Decoder: It uses this encoded information (context vector) to generate the output sequence.
- The model is commonly used in tasks where there is a need to map sequences of varying lengths such as converting a sentence in one language to another or predicting a sequence of future events based on past data i.e time-series forecasting.

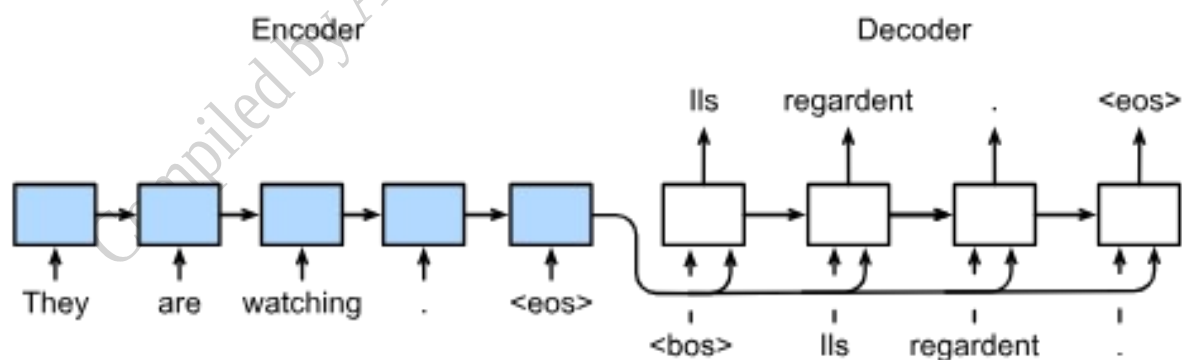


fig. Encoder and Decoder stack in Seq2Seq Model

### Why Sequence-to-Sequence is Needed?

- Traditional models struggle with:
  - Variable-length input/output
  - Retaining long-term dependencies
- Seq2Seq models handle this by using two RNNs (or LSTM/GRU variants):
  - One to encode the input sequence
  - One to decode and generate the output

### Architecture Overview:

A Sequence-to-Sequence (Seq2Seq) model consists of two primary phases: encoding the input sequence and decoding it into an output sequence.

#### 1. Encoding the Input Sequence:

- The encoder processes the input sequence token by token, updating its internal state at each step.
- After the entire sequence is processed, the encoder produces a context vector, a fixed-length representation that summarizes the important information from the input.
  - This context vector is meant to summarize the whole input sentence in a single fixed-length vector.
  - Even if the sentence has 3 words or 30 words, the encoder compresses it into a single vector.
  - That vector will then be used by the decoder to generate the translated/summarized sentence.

#### 2. Decoding the Output Sequence:

- The decoder takes the context vector as input and generates the output sequence one token at a time.
- For example, in machine translation, it can convert the sentence “I am learning” into “Je suis apprenant” sequentially, predicting each token based on the context and previously generated tokens.

#### 3. Teacher Forcing:

- During training, teacher forcing is commonly used.
- Instead of feeding the decoder’s own previous prediction as the next input, the actual target token from the training data is provided.
- This technique accelerates learning and helps the model produce more accurate sequences.

Why use Teacher forcing?

- Because early in training, the model’s predictions are poor and accumulate errors.
- So if we always feed the predicted word back into the decoder, it quickly drifts away from the correct output.
- To prevent that, teacher forcing gives it the right answer at each step : like a teacher guiding it.

Example: Lets say we are translating

- Input: “I am happy”
- Target (Expected Output): “Je suis content”
- Now the decoder generates one word at a time:

| Time Step | What Teacher Forcing Does                    | What Happens Without It             |
|-----------|----------------------------------------------|-------------------------------------|
| t=1       | Input: <SOS> (start of seq.) → predicts “Je” | Input: <SOS> → predicts “Jo”        |
| t=2       | Input: "Je" (from ground truth)              | Input: "Jo" (model’s own output)    |
| t=3       | Input: "suis" (ground truth again)           | Input: “bla...” (prediction drifts) |

### Applications of Seq2Seq Models:

- Machine Translation: Translates text between languages.
- Text Summarization: Summarizes news articles and documents.
- Speech Recognition: Converts speech to text, especially with attention mechanisms.
- Image Captioning: Generates human-readable captions for images by integrating image features with text generation

### Advantages of Seq2Seq Models:

- Flexibility: Can handle tasks like machine translation, text summarization and image captioning with variable-length sequences.
- Handling Sequential Data: Ideal for sequential data like natural language, speech and time series.
- Context Awareness: Encoder-decoder architecture captures the context of the input sequence to generate relevant outputs.
- Attention Mechanism: Focuses on key parts of the input sequence, improving performance, especially for long inputs.

#### Limitations of Seq2Seq Models:

- Computationally Expensive: Requires significant resources to train and optimize.
- Limited Interpretability: Hard to understand the model's decision-making process.
- Overfitting: Prone to overfitting without proper regularization.
- Rare Word Handling: Struggles with rare words not seen during training.
- Limitations of simple Seq2Seq:
  - The fixed-length context vector can bottleneck performance, especially for long sequences.
  - Hard for the decoder to retain details from early parts of the input.

#### Enhancements:

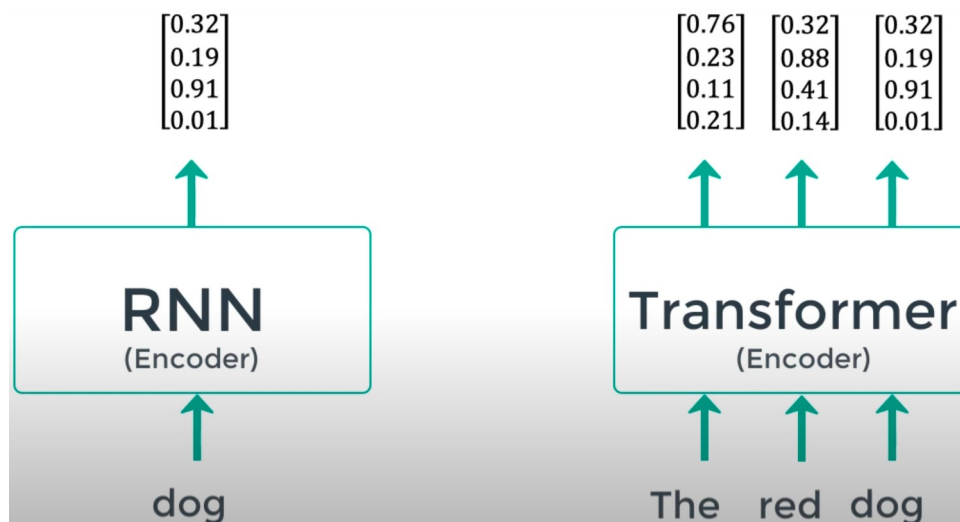
- Attention Mechanism: Allows the decoder to look at all encoder states, not just the last one.
- Bidirectional RNNs: Improve encoder by processing input in both forward and backward directions.
- Transformer models: Completely replace RNNs using self-attention (e.g., BERT, GPT, T5).

### **6.9 The Transformer Architecture- BERT, GPT, Attention:**

- The Transformer is a neural network model designed for machine learning tasks, especially in natural language processing and computer vision.
- It was introduced by Vaswani et al. in their 2017 paper titled "*Attention is All You Need.*"
- They revolutionized NLP by replacing older models like RNNs and LSTMs with a design that relies entirely on a mechanism called **self-attention**.

#### Need for Transformer Model in Machine Learning:

- Traditional models like RNNs (Recurrent Neural Networks) suffer from the vanishing gradient problem which leads to long-term memory loss.
- RNNs process text sequentially meaning they analyze words one at a time.
- While adding more memory cells in LSTMs (Long Short-Term Memory networks) helped address the vanishing gradient issue they still process words one by one.
  - This sequential processing means LSTMs can't analyze an entire sentence at once.
- Transformers process entire sequences at once, rather than step-by-step. This allows them to:
  - Capture global dependencies in the text.
  - Run computations in parallel, making training much faster.
- This is useful where older models work step by step and it helps overcome the challenges seen in models like RNNs and LSTMs.



- Example-1:
  - In the sentence: "XYZ went to France in 2019 when there were no cases of COVID and there he met the president of that country". Here, the word "that country" refers to "France".
  - However RNN would struggle to link "that country" to "France" since it processes each word in sequence leading to losing context over long sentences. This limitation prevents RNNs from understanding the full meaning of the sentence.
- Example-2:
  - Lets see two different sentences:  
She sat by the river bank.  
She deposited money into the bank.
  - Problem with traditional models: they assign single vector for the word "bank" no matter the sentence.
  - In Transformers, the meaning of a word is dynamic — it changes depending on the surrounding words.
  - Through self-attention, the model considers the full sentence, not just local context.
  - Thus, in Transformers, "bank" has two different vector representations depending on the sentence.

### Architecture of Transformer:

Key Components of a transformer:

#### Input Embedding

- every word converted into tokens, then converts tokens into dense vectors.
- Added with Positional Encoding since Transformers have no inherent sequence order (like RNNs).
- Each token is then mapped to a fixed-size dense vector (say, of size 512).

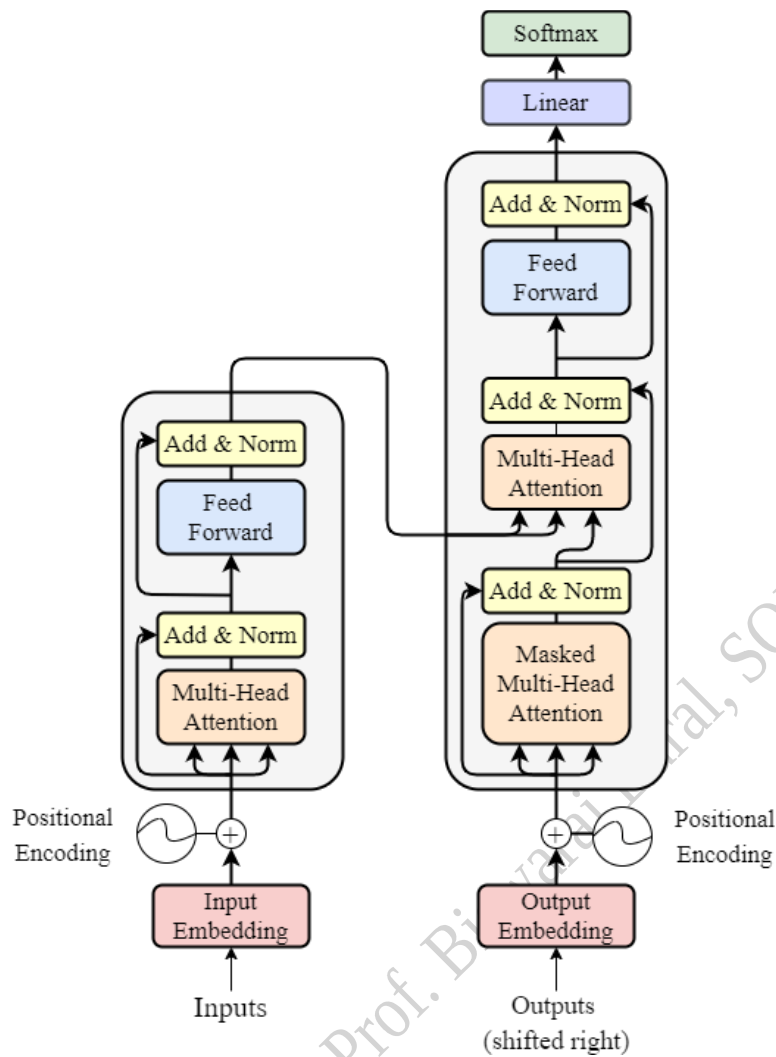


fig. Transformer Architecture (original)

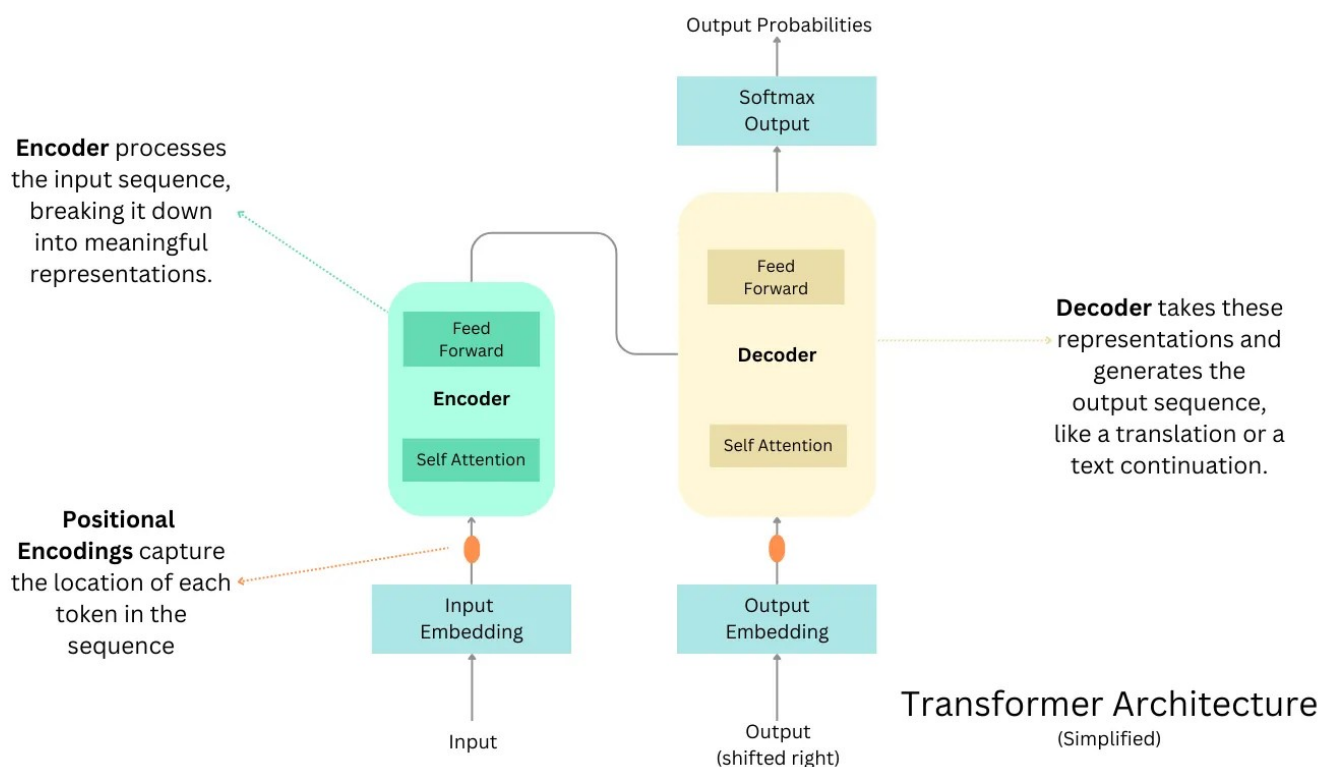


fig. Transformer architecture (simplified)

#### Positional Encoding

- It tells the Transformer which word comes first, second, third, etc.
- Since Transformers see all words at once (not in order), they need extra info to know the position of each word.
  - Transformers don't process words in sequence like RNNs. They look at all tokens at once (parallel processing). So they lack a natural sense of word order. To fix this, we add positional encodings.
- Positional encoding is just a way to add position numbers into word vectors : like a tag that says, "I'm the 3rd word."

#### Multi-Head Self-Attention

- Allows each token to attend to all others in the sequence.
- Multi-head means using multiple attention layers in parallel to capture various relationships.
- Function: Captures contextual meaning of words depending on others in the sentence.

#### Feed-Forward Neural Network (FFN)

- It's a small neural network applied to each word's vector.
- It helps the model learn better patterns by adding extra processing and flexibility.
- Think of it like giving each word a mini upgrade before moving on.

#### Layer Normalization & Residual Connections

- Residuals help avoid vanishing gradients by adding input back to the output of a sub-layer.
- LayerNorm stabilizes and accelerates training.

#### Encoder-Decoder Attention (in Decoder only)

- Allows the decoder to attend to encoder outputs.
- Helps decoder align generated words with input words (e.g., in translation).

#### Output Embedding:

- Output embedding in the decoder is the process of converting previously generated tokens into vectors, so the decoder can understand and process them when predicting the next word.

#### Output Linear Layer + Softmax

- Final dense layer maps decoder output to vocabulary size.
- Softmax gives probabilities over all words.
- Transformers predict the next token or label probabilities based on learned patterns and input context.
- The Transformer outputs a vector for each input token.
- Each output vector has the same dimensionality as the embedding size (Eg. 512-d).
- So, for 100 tokens, you get 100 vectors, each of size 512.

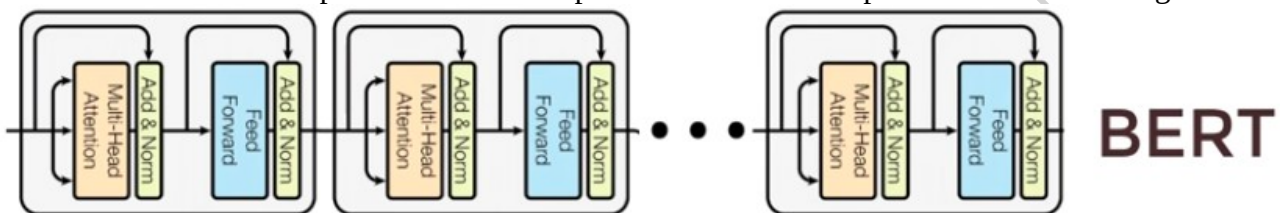
#### **BERT (Bidirectional Encoder Representation from Transformers):**

- BERT is a pre-trained language model developed by Google in 2018 that understands the context of words in a sentence by looking both left and right (bidirectional).
- It is built entirely using the Transformer Encoder architecture.
- Properties of BERT:
  - Deeply bidirectional

- Pre-trained on massive corpus (Wikipedia + BooksCorpus)
- General-purpose : just fine-tune on a downstream task with minimal changes
- Input length typically  $\leq 512$  tokens
- Application of BERT:
  - Text classification
  - Sentence pair classification (e.g., paraphrase detection)
  - Named Entity Recognition (eg. people, location, organization, dates etc)
  - Question Answering
  - Text summarization

### How does BERT works?

- Uses Transformer Encoder (Only the left half):
  - Unlike GPT, which uses only the decoder, BERT uses stacked Transformer encoders.
  - Each encoder processes tokens in parallel and builds deep contextual embeddings.



- Bidirectional Attention:
  - BERT looks at the entire sentence at once (left and right context) for each word.
  - This allows it to learn deeper meaning and relationships between words.
- Pre-training Objectives:
 

BERT is trained on two main self-supervised tasks:

  1. Masked Language Modeling (MLM):
    - It is a training strategy used to help BERT learn language better by understanding the context of words from both left and right.
    - Randomly mask 15% of words in the input sentence.
    - Replace those words with a special token: [MASK].
    - The model's task is to predict the original word that was masked, using the surrounding words (left and right).
    - Example: "I took my [MASK] to the vet." → dog
      - BERT uses the context from both sides: "I took my" (before) and "to the vet" (after) to guess the missing word.; Target (correct answer): "dog"
  2. Next Sentence Prediction (NSP)
    - Given two sentences A and B:
    - Model predicts if B follows A in original text.
    - Helps learn sentence relationships (useful for QA).
- Input Representation:
 

Before giving text to BERT, it needs to be converted into numbers: called embeddings. BERT uses 3 types of embeddings, and adds them together for each word. BERT input includes:

  - Token embeddings (from vocabulary):
    - These are the normal word embeddings (like word2vec or GloVe): each word is turned into a vector from the vocabulary.

- Example: ["I", "love", "cats"] → [[0.12, -0.4, ...], [0.89, 0.01, ...], [0.34, 0.5, ...]]
- Segment embeddings (to distinguish sentence A and B):
  - BERT is trained on sentence pairs.
  - So it needs to know which word belongs to Sentence A or Sentence B.
  - Sentence A → Segment ID = 0
  - Sentence B → Segment ID = 1
  - This helps BERT learn relationships between two sentences.
- Position embeddings (to capture word order):
  - Transformers don't understand order by default.
  - So BERT adds position information to each token:
  - Word at position 0, 1, 2... gets a unique embedding.
- Final input is: [CLS] Sentence A [SEP] Sentence B [SEP] ie sum of 3 embeddings
  - For each word in the input, BERT adds:
    - Token Embedding + Segment Embedding + Position Embedding
  - This sum becomes the input to the transformer layers.
  - If you're giving two sentences:
  - "The cat sat." and "It was sleepy."
  - BERT input looks like:
    - [CLS] The cat sat . [SEP] It was sleepy . [SEP]
    - [CLS] → Special token for classification (goes at the start)
    - BERT uses the final hidden vector of the [CLS] token as the summary of the whole input, and passes it to a classifier.
  - [SEP] → Special token to separate sentences
- Output from BERT
  - The output is a sequence of contextual vectors (one per token): shape [sequence\_length, hidden\_size].
  - The special [CLS] token at the start is used for classification tasks.
  - Outputs are fine-tuned for tasks like:
    - Sentiment Analysis
    - Question Answering (e.g., SquAD-dataset commonly used)

## **GPT(Generative Pre-trained Transformer):**

### What is GPT?

- GPT is a type of Transformer-based language model developed by OpenAI.
- It is designed to generate human-like text given some input, making it powerful for a wide range of NLP tasks like:
  - Text generation
  - Summarization
  - Translation
  - Question answering
  - Chatbots
- Why Generative? Because GPT can:
  - Generate new text
  - Continue writing a story

- Simulate dialogue
- Write code or summarize paragraphs

### Core Idea behind GPT:

GPT Follows two main steps: pre-training and Fine-Tuning.

- Pre-training:
  - It learns language patterns from a large amount of text.
  - Task: Predict the next word in a sentence (called causal language modeling).
  - Example: "The cat sat on the \_\_\_\_" → learns to predict "mat".
- Fine-tuning:
  - GPT can be fine-tuned on specific tasks or datasets (e.g., medical QA, legal summarization).
  - This allows GPT to adapt to domain-specific needs.

### Architecture of GPT:

GPT is based entirely on the **decoder block** of the original Transformer architecture.

Key Components:

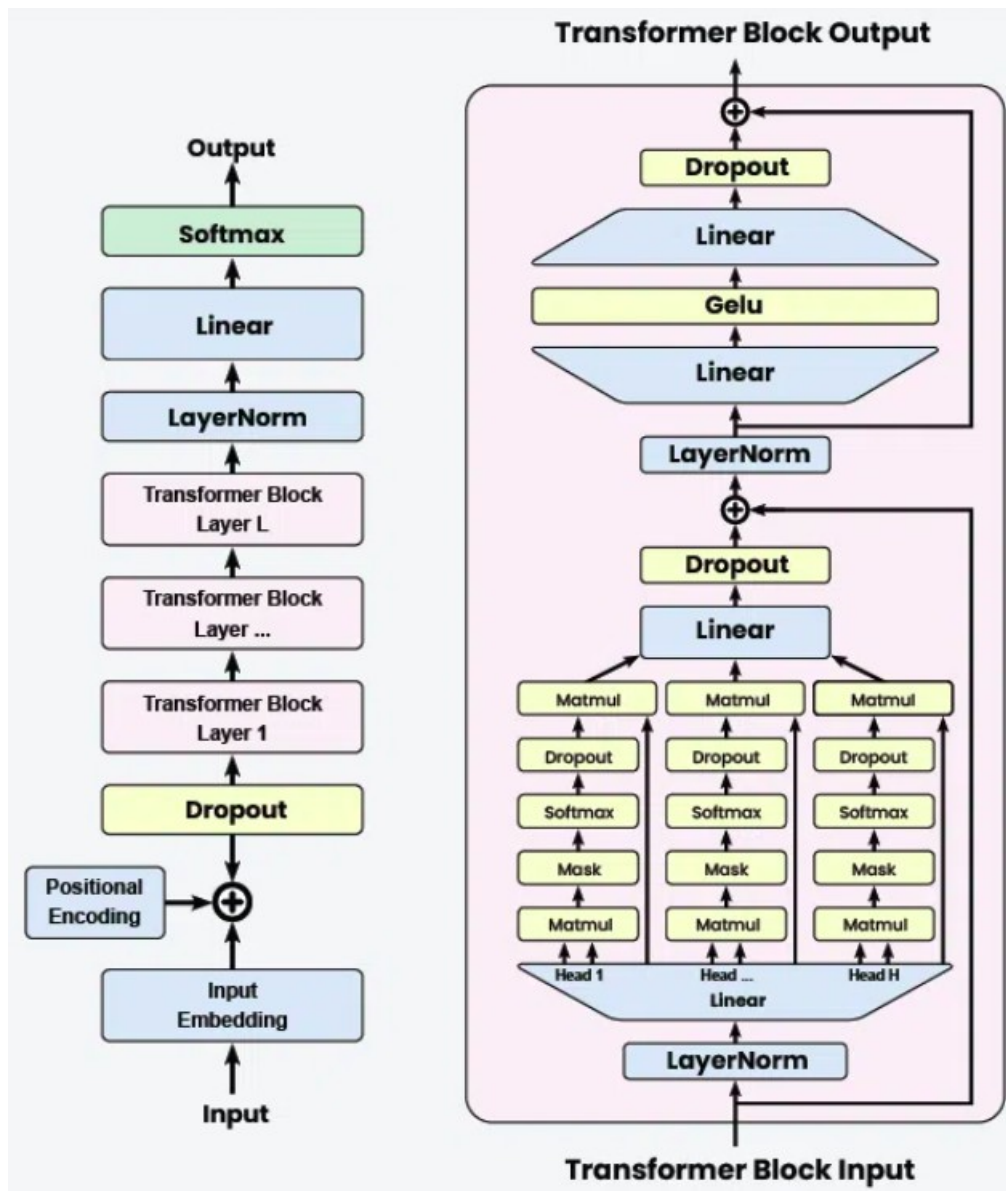
- Tokenization:
  - Breaks input text into tokens (subwords or characters).
  - Example: "playing" → ['play', '##ing']
- Input Embeddings:
  - Converts each token into a dense vector.
  - Positional encodings are added to maintain word order.
- Transformer Decoder Blocks (repeated multiple times):
 

Each block contains:

  - Masked Multi-Head Self-Attention: So the model can only look leftward (past tokens).
  - Layer Normalization
  - Feed-Forward Neural Network (FFN)
  - Residual Connections
- Output Layer:
  - A softmax layer over vocabulary predicts the next token.
- Example Flow:
 

Input: "The capital of France is"

  - Tokenize
  - Embed
  - Apply transformer decoder blocks
  - Softmax predicts: "Paris"



### How GPT Learns:

#### GPT Uses:

- Unsupervised learning for pretraining (from web data, books, etc.).
- Supervised fine-tuning on labeled datasets if needed.
- Reinforcement Learning with Human Feedback (RLHF) in newer versions (e.g., ChatGPT) to make responses more useful and safe.

### Input and Output:

- Input: Tokenized text sequence.
- Output: A sequence of probabilities over vocabulary.
  - Final output = the most probable next word(s).